信息与软件工程学院

编译技术

主讲教师：陈安龙

第7章 代码优化与生成

- 7.1 代码优化
- 7.2 基本块和流图
- 7.3 基本块的优化
- 7.4 代码优化示例
- 7.5 目标代码生成
- 7.6 寄存器分配算法

7.1 代码优化

代码优化是编译器设计中的一个重要环节，旨在提高生成代码的执行效率、减少代码的空间占用和提高程序的整体性能。代码优化可以在编译的多个阶段进行，包括源代码优化、中间代码优化和目标代码优化。

源代码优化

在源代码级别进行的优化，通常由程序员手动进行，或通过一些静态分析工具进行。常见的源代码优化包括：

① 消除冗余代码：删除不必要的计算或无用的代码。

② 使用高效的数据结构：选择合适的数据结构以提高访问效率。

③ 循环优化：通过改写循环结构来提高效率。

中间代码优化

在编译器生成中间代码后进行的优化，通常包括：

① 常量折叠：在编译时计算常量表达式的值。

② 死代码消除：删除不会被执行的代码。

③ 公共子表达式消除：识别并重用相同的表达式计算结果。

目标代码优化

在生成目标代码后进行的优化，通常包括：

① 指令选择优化：选择更高效的机器指令。

② 寄存器分配优化：优化寄存器的使用，减少内存访问。

③ 指令调度：重新排列指令以提高并行性和减少延迟。

7.2 基本块和流图

程序基本块

① 只有一个入口点：程序执行到基本块的开始点，不能从基本块外部进入。

② 只有一个出口点：程序执行到基本块的结束点，不能在基本块内部跳转到其他地方。

③ 基本块内的指令是顺序执行的，没有分支或跳转。

划分基本块

把三地址语句序列划分成基本块

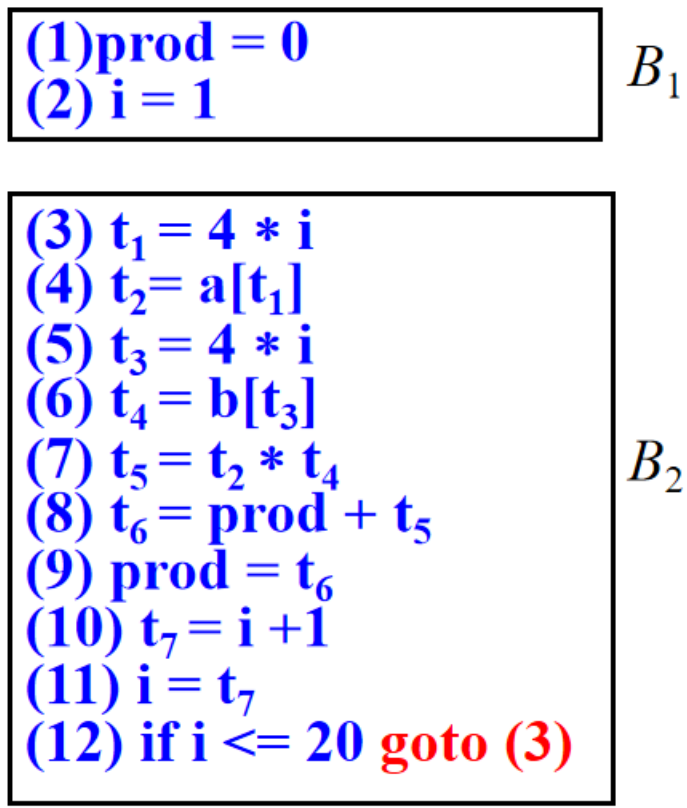
(1) 首先确定所有的入口语句

- 序列的第一个语句是入口语句
- 能由条件转移语句或无条件转移语句转到的语句是入口语句
- 紧跟在条件转移语句或无条件转移语句后面的语句是入口语句

(2) 每个入口语句到下一个入口语句之前（或到程序结束）的语句序列构成一个基本块

划分基本块

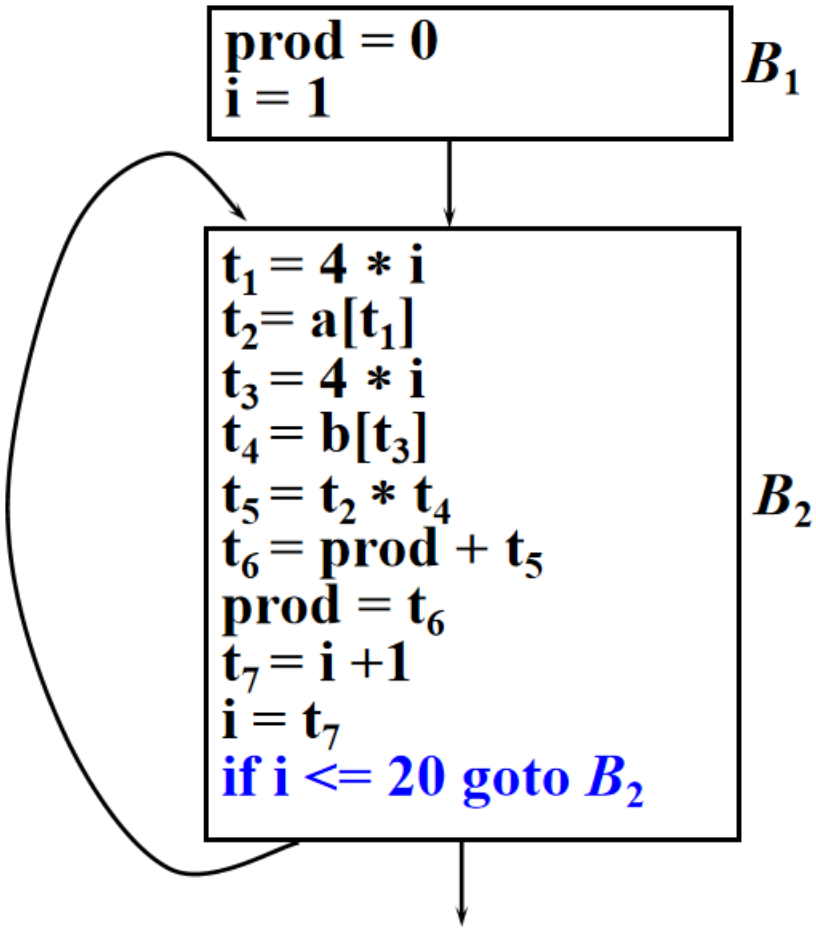
(1) prod = 0
(2) i = 1
(3) $t_1 = 4 * i$
(4) $t_2 = a[t_1]$
(5) $t_3 = 4 * i$
(6) $t_4 = b[t_3]$
(7) $t_5 = t_2 * t_4$
(8) $t_6 = prod + t_5$
(9) $prod = t_6$
(10) $t_7 = i + 1$
(11) $i = t_7$
(12) if i <= 20 goto (3)



程序流图

- 程序流图（Control Flow Graph, CFG）是编译器设计中的重要概念，用于表示程序中控制流的结构。它为编译器的优化、分析和代码生成提供了基础。程序流图通常由以下部分构成：
  ① 节点：每个节点表示一个基本块，基本块内的代码是顺序执行的。
  ② 边：边表示控制流的转移，通常由条件语句、循环和跳转语句引起。

- 什么是循环?
  - 所有结点是强连通的
  - 唯一的循环入口
- 外循环和内循环



程序流图的构建方法

构建程序流图的步骤通常包括：

① 识别基本块：将程序分解为基本块，识别每个基本块的入口和出口。

② 建立控制流关系：根据程序中的控制结构（如条件语句、循环、跳转等）建立基本块之间的边。

③ 绘制图形：将基本块表示为节点，控制流关系表示为边，绘制出程序流图。

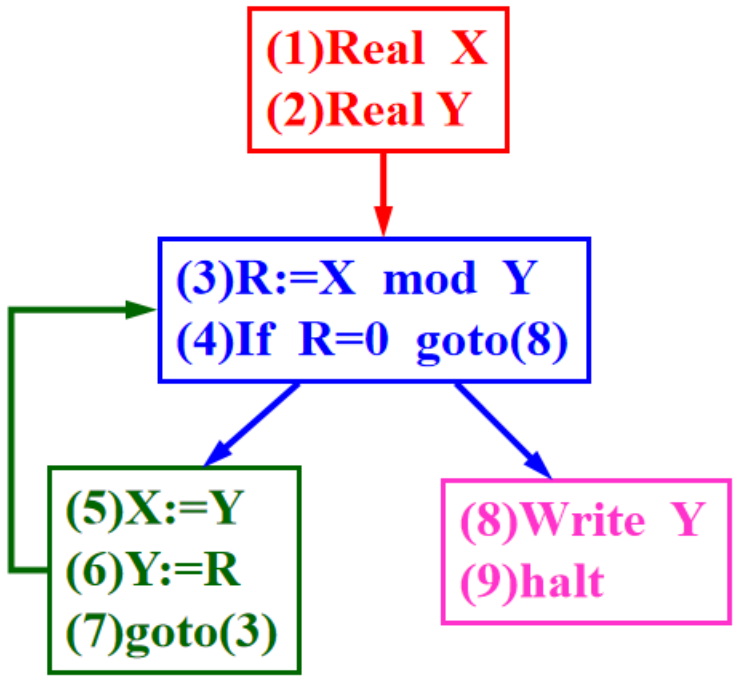
举例:考察下面的三地址代码程序

（1）Read X
（2）Read Y
} B1

（3）R:=X mod Y
（4）if R=0 goto (8)
} B2

（5）X:=Y
（6）Y:=R
（7）goto(3)
} B3

（8）write Y
（9）halt
} B4



7.3 有向无环图(DAG)

在编译技术中，基本块的有向无环图（Directed Acyclic Graph, DAG）是一种用于表示基本块内部计算关系的图形结构。DAG图能够有效地表示表达式的计算过程，帮助编译器进行优化，如公共子表达式消除、指令选择等。

DAG图的构成

DAG图通常由以下部分构成：

① 节点：表示计算的操作数或结果。

② 边：表示计算之间的依赖关系，指示数据流向。

③ DAG图的特点是没有环路，这意味着从一个节点出发，无法回到该节点。

DAG图的用途

DAG图在编译器中的主要用途包括：

① 公共子表达式消除：通过识别相同的子表达式，减少冗余计算。

② 优化指令选择：为目标代码生成提供更高效的指令选择。

③ 数据流分析：支持数据流分析技术，帮助编译器进行优化。

DAG图的构建方法

构建基本块的DAG图的步骤通常包括：

① 识别基本块：将程序分解为基本块，识别每个基本块的入口和出口。

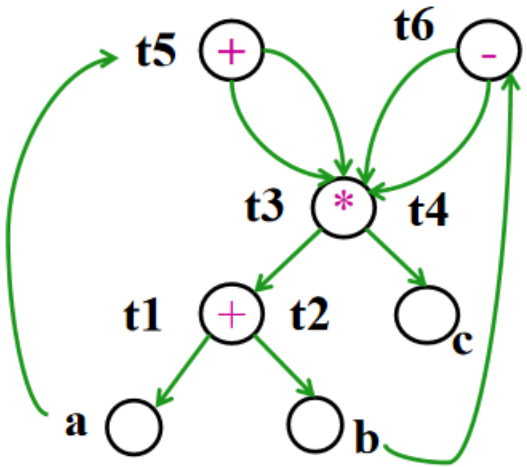
② 分析表达式：分析基本块内的表达式，识别操作数和运算。

③ 建立DAG图：根据操作数和运算建立DAG图，添加节点和边。

创建DAG示例

假设有以下基本块

① t1 = a + b
② t2 = a + b
③ t3 = t1 * c
④ t4 = t2 * c
⑤ t5 = t3 + t4
⑥ t6 = t3 - t4
⑦ a = t5
⑧ b = t6



构建DAG的过程如下：

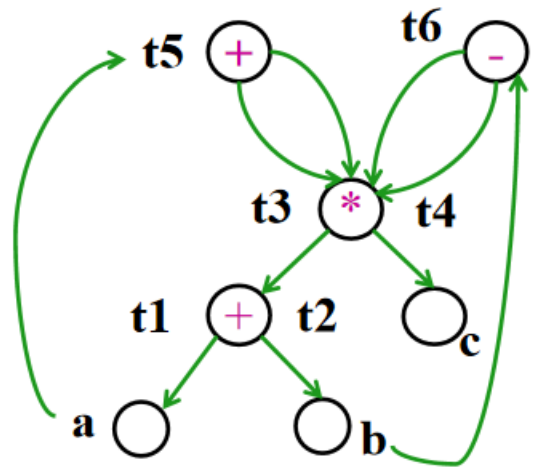
① 创建节点 +，子节点为 a 和 b，并让 t1 和 t2 指向这个节点（公共子表达式消除）。
② 创建节点 *，子节点为 t1（即 a + b）和 c，并让 t3 和 t4 指向这个节点（公共子表达式消除）。
③ 创建节点 +，子节点为 t3 和 t4，并让 t5 指向这个节点。
④ 创建节点 -，子节点为 t3 和 t4，并让 t6 指向这个节点。
⑤ 最后，a 指向 t5 的结果，b 指向 t6 的结果。

优化DAG

- **公共子表达式消除：**如果多个表达式在DAG中指向同一个节点，那么这些表达式是公共的，可以消除重复计算。
- **复制传播：**如果一个变量是另一个变量的复制，可以在使用它的地方直接替换为原变量。
- **死代码消除：**如果DAG中的某个节点没有后继节点，并且对应的变量在后面没有被使用，那么这个节点对应的代码可以被删除。

优化DAG示例

① t1 = a + b
② t3 = t1 * c
③ a = t3 + t3
④ b = t3 - t3



在优化过程中，消除了公共子表达式 t2 = a + b 和 t4 = t2 * c，因为它们与 t1 和 t3 相同。可以进一步优化，因为 t3 - t3 总是等于0，所以 b 可以直接赋值为0，而 t3 + t3 等于 2 * t3，所以 a 可以直接赋值为 2 * t3。

最终优化后的代码：

① t1 = a + b
② t3 = t1 * c
③ a = 2 * t3
④ b = 0

7.4 基本块优化

■ 局部公共子表达式消除

公共子表达式是在一个基本块中多次计算的表达式。如果一个表达式之前已经计算过，并且自那以后它的操作数没有改变，那么这个表达式就不需要再次计算。

优化前的三地址代码：

```
t1 = a * b
t2 = c * d
t3 = a * b
t4 = t2 + t3
t5 = t1 - t4
```

优化后的三地址代码：

```
t1 = a * b
t2 = c * d
t4 = t2 + t1
t5 = t1 - t4
```

7.4 基本块优化

■ 死代码消除（Dead Code Elimination, DCE）

死代码是指不可能执行的代码，或者执行了也不会对程序的输出产生影响的代码。

优化前的三地址代码：

```
t1 = x + y
t2 = u * v
t3 = t1 + t2
t4 = z - t3
t5 = t1 - t2
t6 = t5 + t4
t7 = t2 * t6
t8 = t1 * t2
```

如果`t8`的值在后面的代码中没有被使用，
那么`t8`的计算就是死代码。

优化后的三地址代码：

```
t1 = x + y
t2 = u * v
t3 = t1 + t2
t4 = z - t3
t5 = t1 - t2
t6 = t5 + t4
t7 = t2 * t6
```

7.4 基本块优化

■ 强度削减（Strength Reduction）

强度削弱是指用一种计算成本较低的操作来替换一种计算成本较高的操作。

优化前的三地址代码：

```
t1 = i * 8
t2 = a[t1]
t3 = j * 1024
t4 = b[t3]
```

优化后的三地址代码：

```
t1 = i << 3 // 用左移替代乘以8
t2 = a[t1]
t3 = j << 10 // 用左移替代乘以1024
t4 = b[t3]
```

7.4 基本块优化

- 代码移动（Code Motion）

循环不变代码是指在循环中，其值不会改变的代码。这种代码可以被移动到循环外部执行，以减少循环的执行时间。

优化前的三地址代码：

```
loop: t1 = y * z
      t2 = x + t1
      t3 = a[i]
      t4 = t2 * t3
      a[i] = t4
      i = i + 1
      if i < n goto loop
```

优化后的三地址代码：

```
      t1 = y * z
loop: t2 = x + t1
      t3 = a[i]
      t4 = t2 * t3
      a[i] = t4
      i = i + 1
      if i < n goto loop
```

7.4 基本块优化

- 循环不变代码外提（Loop Invariant Code Motion）

循环不变代码是指在循环中，其值不会改变的代码。这种代码可以被移动到循环外部执行，以减少循环的执行时间。

优化前的三地址代码：

```
loop: t1 = x + y
      t2 = t1 * z
      t3 = a[i]
      t4 = t2 * t3
      a[i] = t4
      i = i + 1
      if i < n goto loop
```

优化后的三地址代码：

```
      t1 = x + y
      t2 = t1 * z
loop: t3 = a[i]
      t4 = t2 * t3
      a[i] = t4
      i = i + 1
      if i < n goto loop
```

代码优化示例

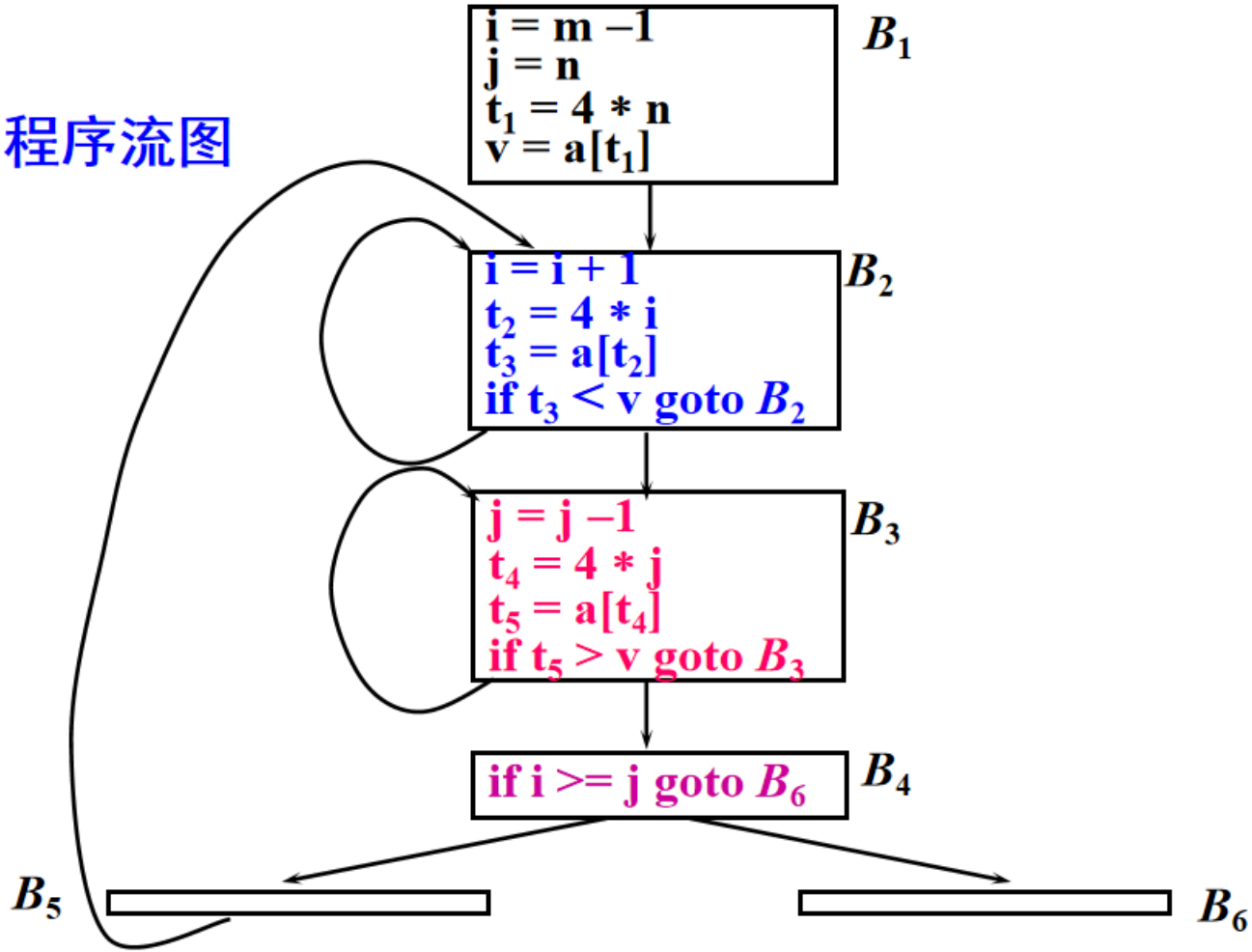
假设如下程序实例

```
i = m –1; j = n; v = a[n];
while (1) {
    do i = i +1; while(a[i]<v);
    do j =j –1;while (a[j]>v);
    if (i >= j) break;
    x=a[i]; a[i]=a[j]; a[j]=x;
}
x=a[i]; a[i]=a[n]; a[n]=x;
```

```
(1)  i = m –1
(2)  j = n
(3)  t1 = 4 * n
(4)  v = a[t1]
(5)  i = i + 1
(6)  t2 = 4 * i
(7)  t3 = a[t2]
(8)  if t3 < v goto (5)
(9)    j = j –1
(10) t4 = 4 * j
(11) t5 = a[t4]
(12) if t5>v goto (9)
(13) if i >=j goto (23)
(14) t6 = 4 * i
(15 ) x = a[t6]
. . .
```

7.5 代码优化

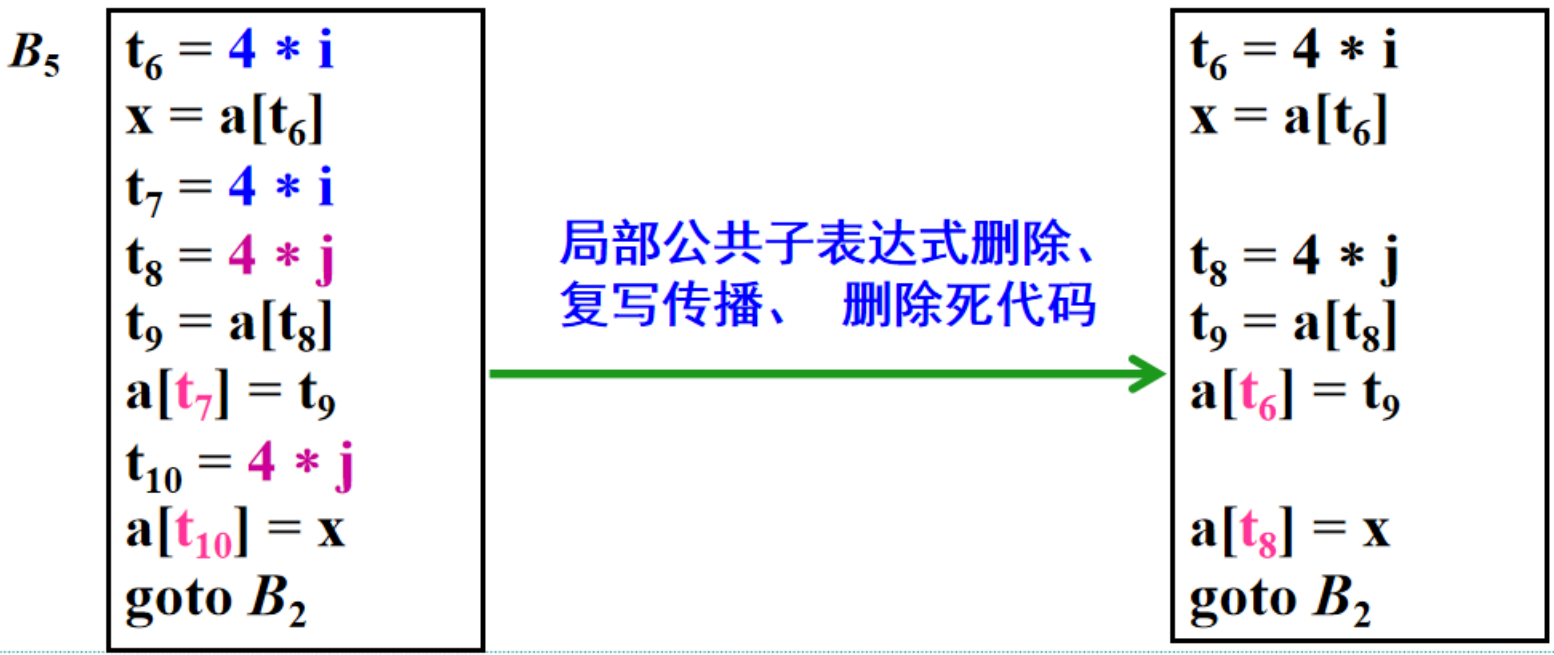
- 程序流图

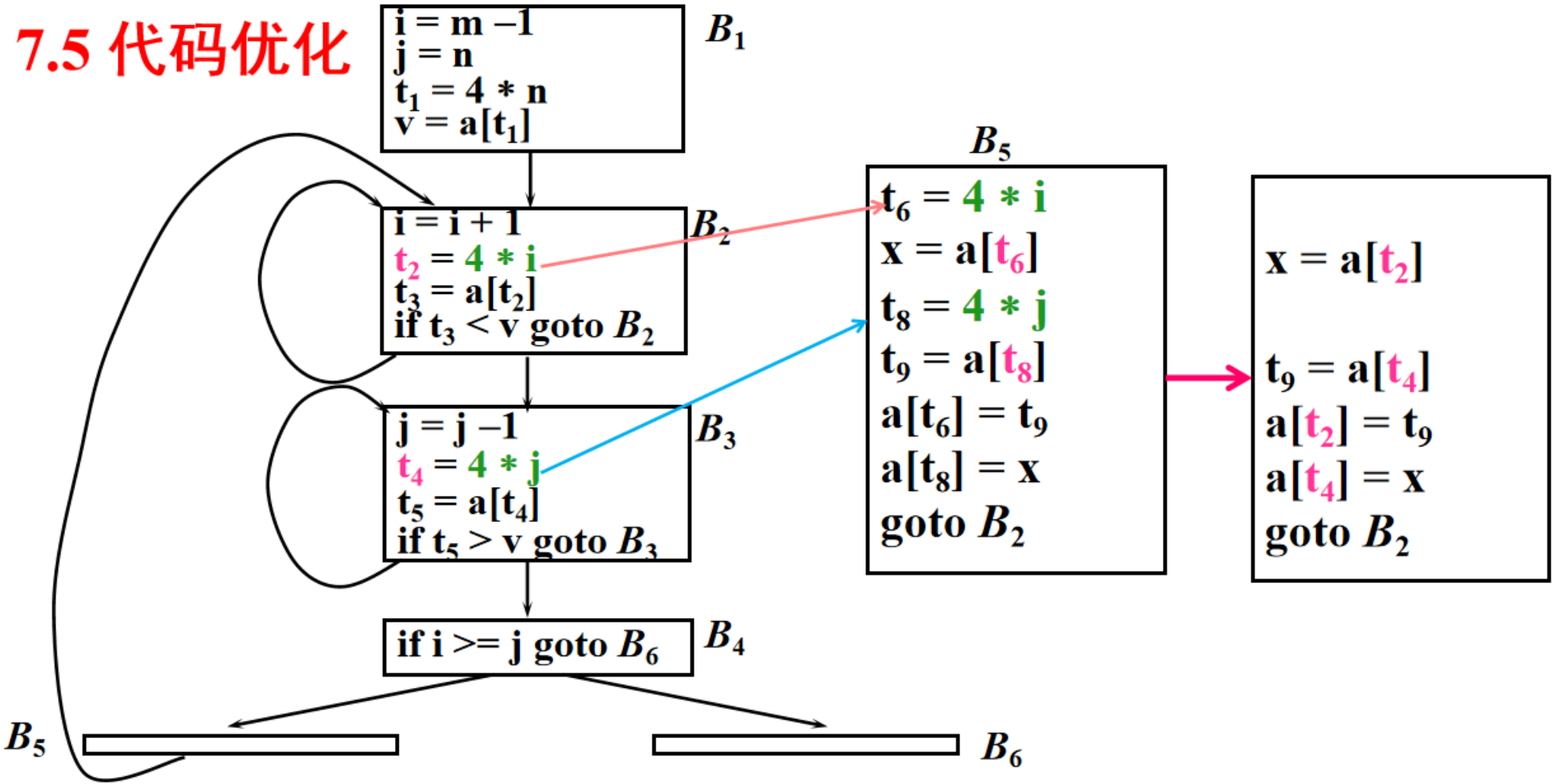

i = m –1
j = n
t1 = 4 * n
v = a[t1]
B1
i = i + 1
t2 = 4 * i
t3 = a[t2]
if t3 < v goto B2
B2
j = j –1
t4 = 4 * j
t5 = a[t4]
if t5 > v goto B3
B3
if i >= j goto B6
B4
B5
B6

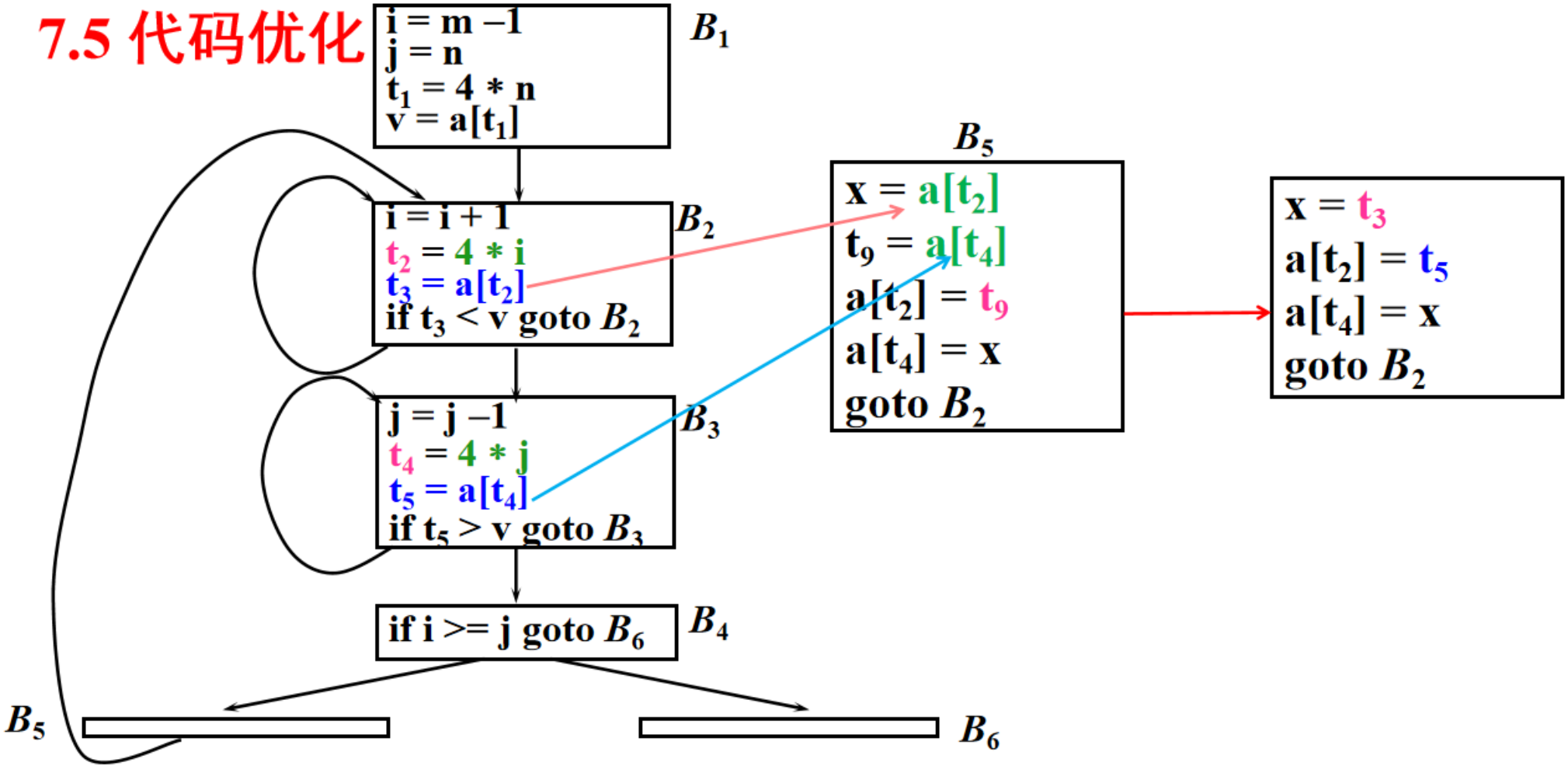
7.5 代码优化

公共子表达式删除

B_5 x=a[i]; a[i]=a[j]; a[j]=x;

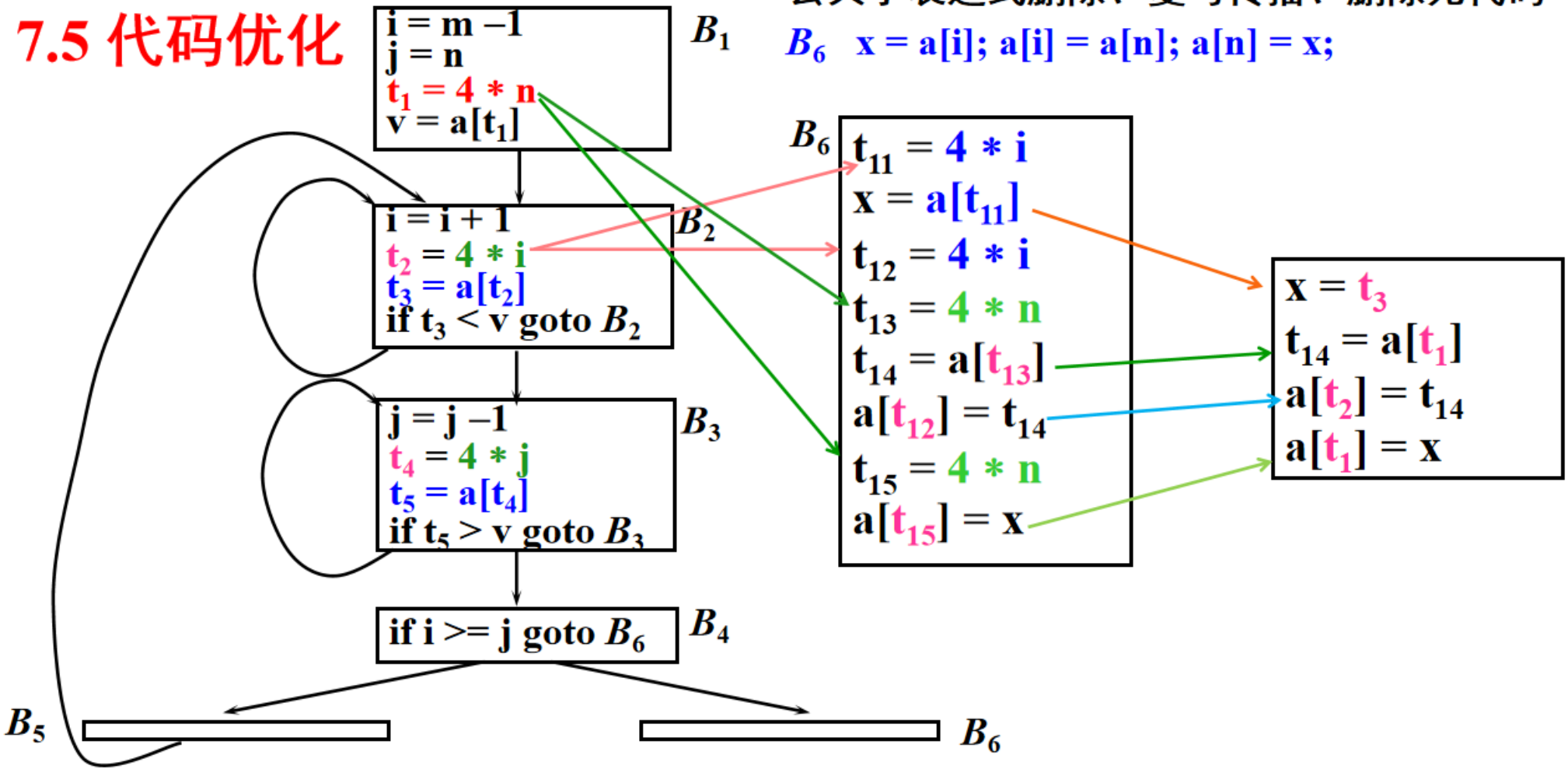



i = m –1
j = n
t1 = 4 * n
v = a[t1]
B1
i = i + 1
t2 = 4 * i
t3 = a[t2]
if t3 < v goto B2
B2
j = j –1
t4 = 4 * j
t5 = a[t4]
if t5 > v goto B3
B3
if i >= j goto B6
B4
B5
t6 = 4 * i
x = a[t6]
t8 = 4 * j
t9 = a[t8]
a[t6] = t9
a[t8] = x
goto B2
B5
x = a[t2]
t9 = a[t4]
a[t2] = t9
a[t4] = x
goto B2
B5
B6


i = m –1
j = n
t1 = 4 * n
v = a[t1]
B1
i = i + 1
t2 = 4 * i
t3 = a[t2]
if t3 < v goto B2
B2
j = j –1
t4 = 4 * j
t5 = a[t4]
if t5 > v goto B3
B3
if i >= j goto B6
B4
B5
B6
x = a[t2]
t9 = a[t4]
a[t2] = t9
a[t4] = x
goto B2
B5
x = t3
a[t2] = t5
a[t4] = x
goto B2

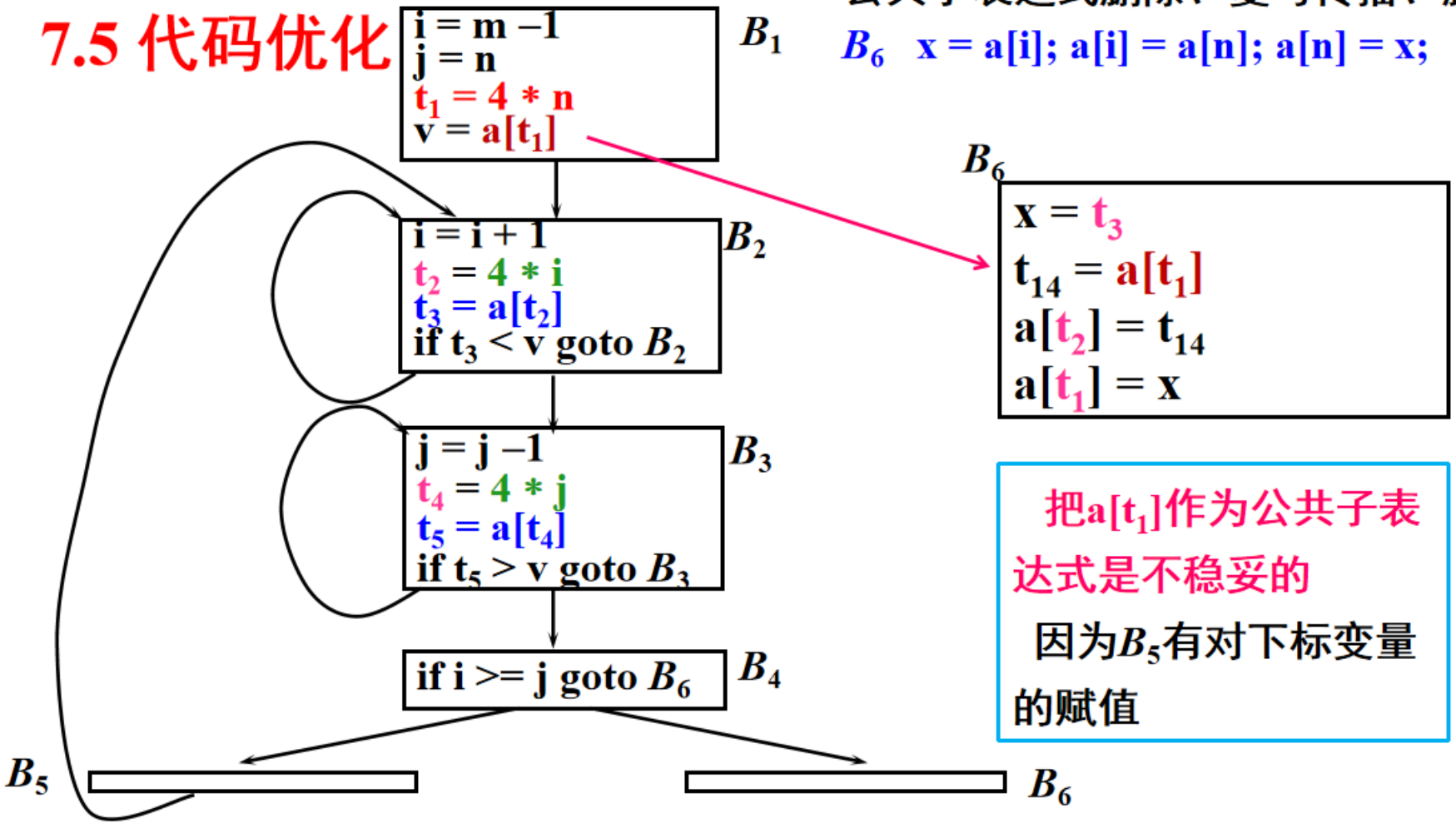
公共子表达式删除、复写传播、删除死代码

B_6 x = a[i]; a[i] = a[n]; a[n] = x;



公共子表达式删除、复写传播、删除死代码

B_6 x = a[i]; a[i] = a[n]; a[n] = x;



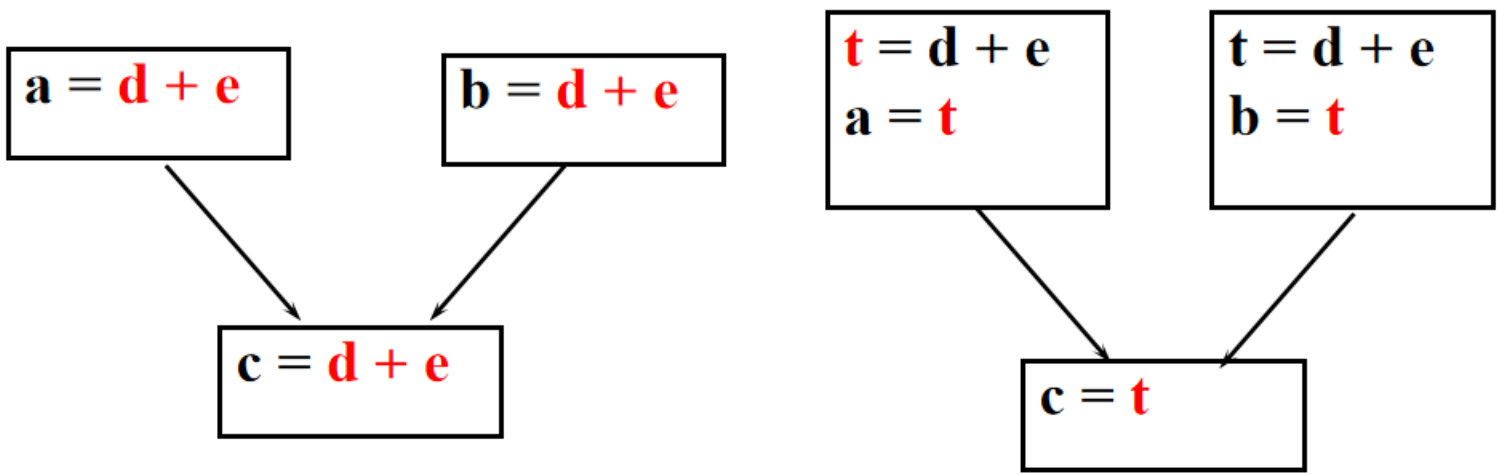
把$a[t_1]$作为公共子表达式是不稳妥的

因为B_5有对下标变量的赋值

7.5 代码优化

复写传播

- 复写语句：形式为f = g的赋值
- 优化过程中会大量引入复写



删除局部公共子表达式期间引进复写

7.5 代码优化

复写传播

- 复写语句：形式为$f = g$的赋值
- 优化过程中会大量引入复写
- 复写传播变换的做法是在复写语句$f = g$后，尽可能用g代表f

B_5

$x = t_3$
$a[t_2] = t_5$
$a[t_4] = x$
goto B_2

$x = t_3$
$a[t_2] = t_5$
$a[t_4] = t_3$
goto B_2

7.5 代码优化

死代码删除

- 死代码是指计算的结果决不被引用的语句
- 一些优化变换可能会引起死代码

例如右边代码，优化代码如下：

```
#include <stdio.h>
int main() {
    int a = 10;
    int b = 20;
    int c = a + b;
    printf("The result is %d\n", c);
    return 0;
}
```

```
#include <stdio.h>
int main() {
    int a = 10;
    int b = 20;
    int c = a + b;
    int d = 30; // 死代码：d的值被赋值但从未使用
    int e = 40; // 死代码：e的值被赋值但从未使用
    int f;

    if (c > 100) {
        f = 50; // 死代码：这行代码永远不会执行
    }
    printf("The result is %d\n", c);
    return 0;
}
```

7.5 代码优化

死代码删除

- 死代码是指计算的结果决不被引用的语句
- 一些优化变换可能会引起死代码

例如： 复写传播可能会引起死代码删除

B5

$x = t_3$ $a[t_2] = t_5$ $a[t_4] = x$ goto B_2	$x = t_3$ $a[t_2] = t_5$ $a[t_4] = t_3$ goto B_2	$a[t_2] = t_5$ $a[t_4] = t_3$ goto B_2

7.5 代码优化

代码外提

- 代码外提是循环优化的一种
- 循环优化的其他重要技术
  - ✓ 归纳变量删除
  - ✓ 强度削弱

例：while (i <= limit – 2) …

代码外提后变换成

```
t = limit – 2;
while (i <= t ) …
```

7.5 代码优化

强度削弱和归纳变量删除

① j和t_4的值步伐一致地变化，这样的变量叫做归纳变量

② 在循环中有多个归纳变量时，也许只需要留下一个

③ 这个操作由归纳变量删除过程来完成

④ 对本例可以先做强度削弱它给删除归纳变量创造机会

B_3

$j = j - 1$

$t_4 = 4 * j$

$t_5 = a[t_4]$

if $t_5 > v$ goto B_3

7.5 代码优化

强度削弱和归纳变量删除

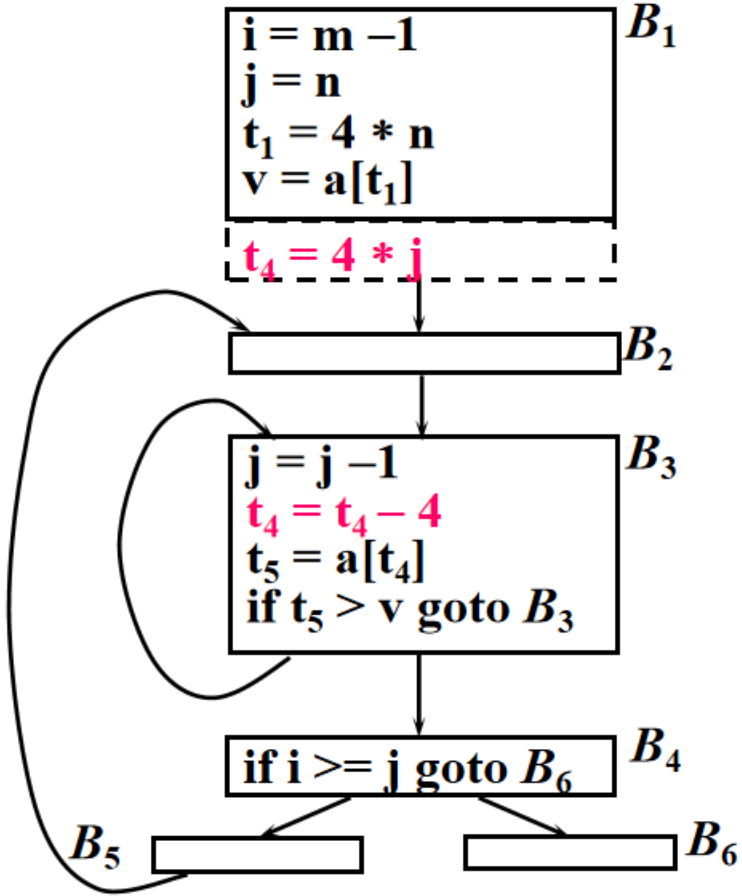
B_3

```
j = j –1
t4 = 4 * j
t5 = a[t4]
if t5 > v goto B3
```

除第一次外，

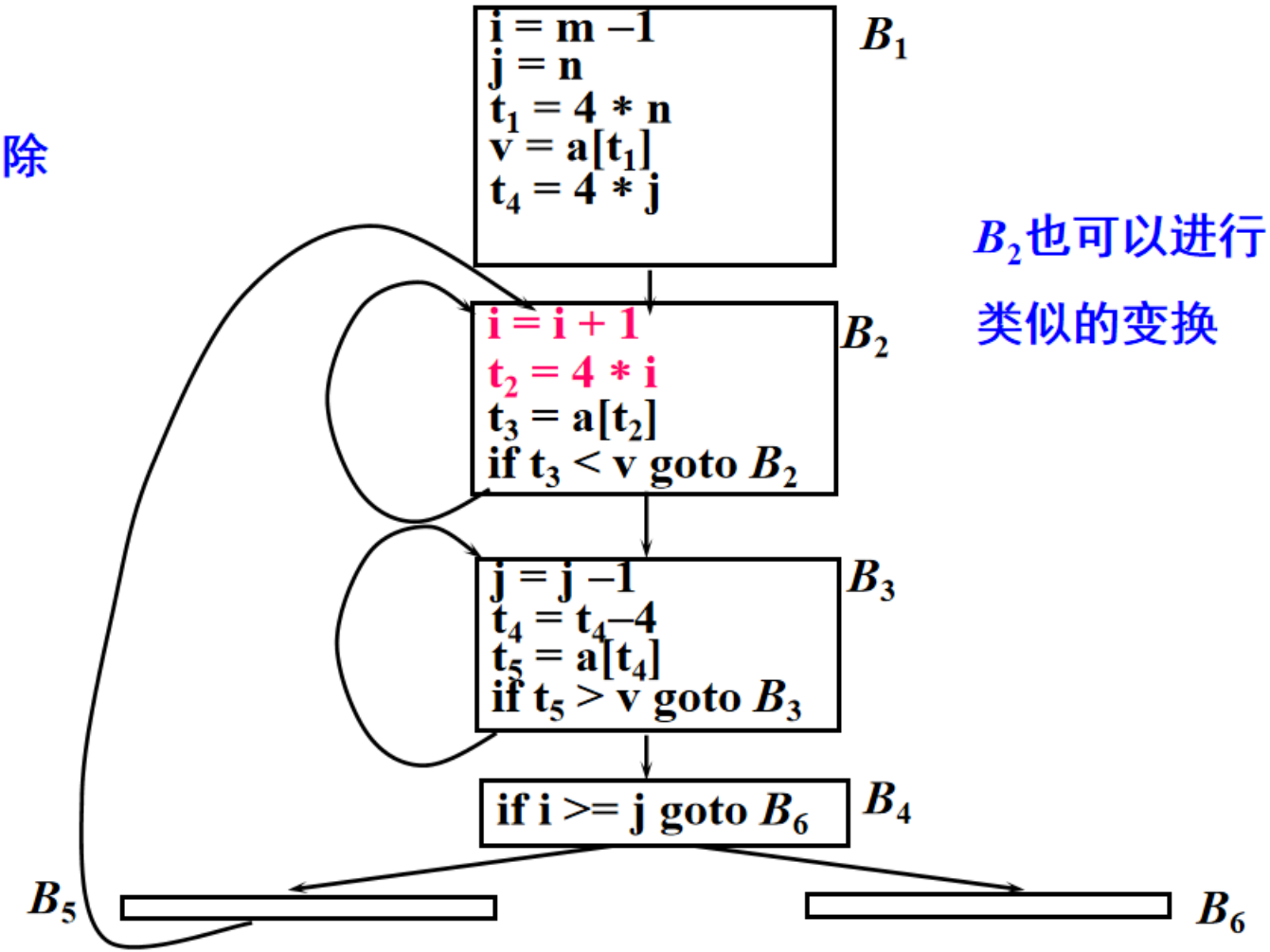
$t_4 == 4 * j$在B_3的入口一定保持

在j = j –1后，关系$t_4 == 4 * j + 4$也保持



7.5 代码优化

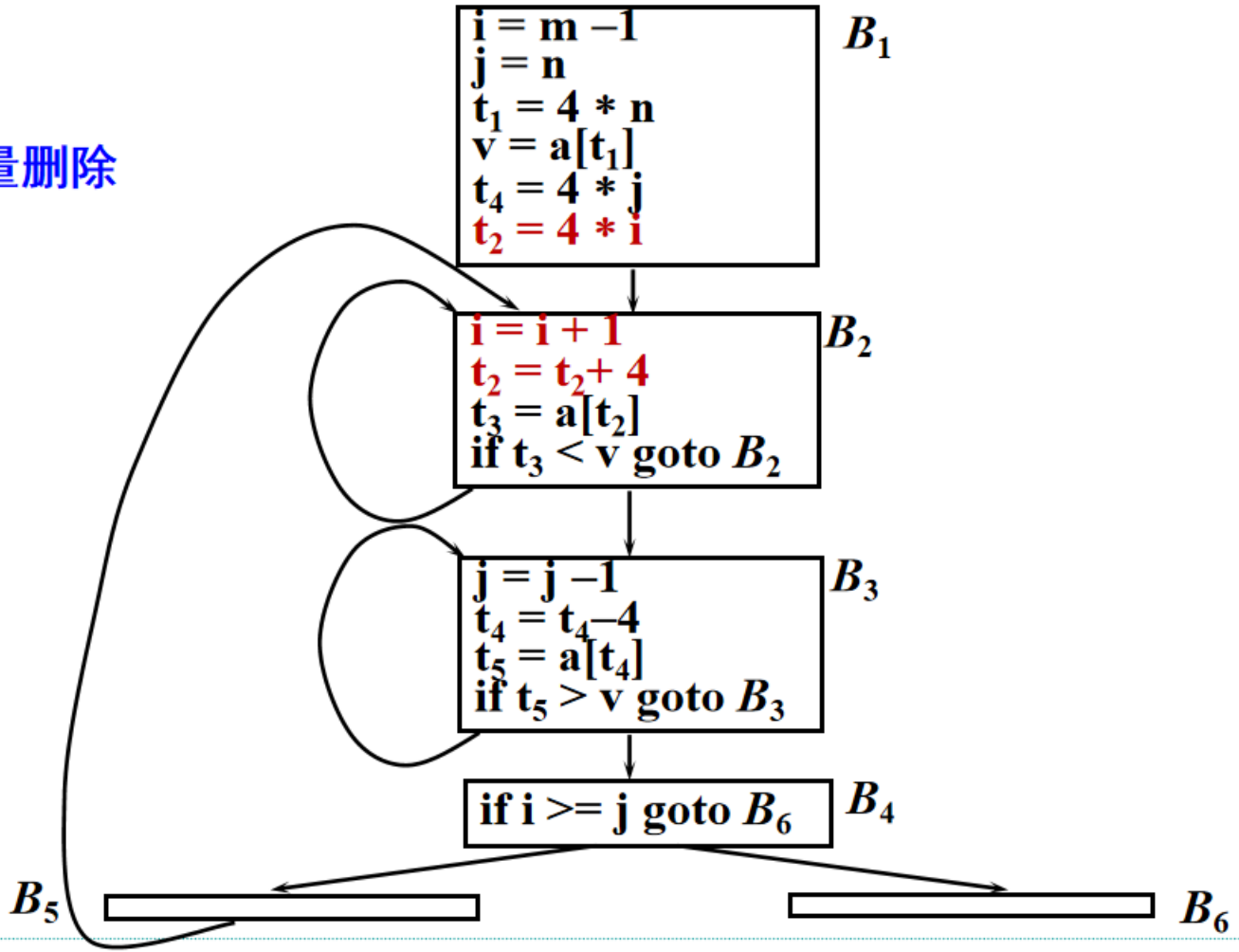
强度削弱和归纳变量删除



B_2也可以进行类似的变换

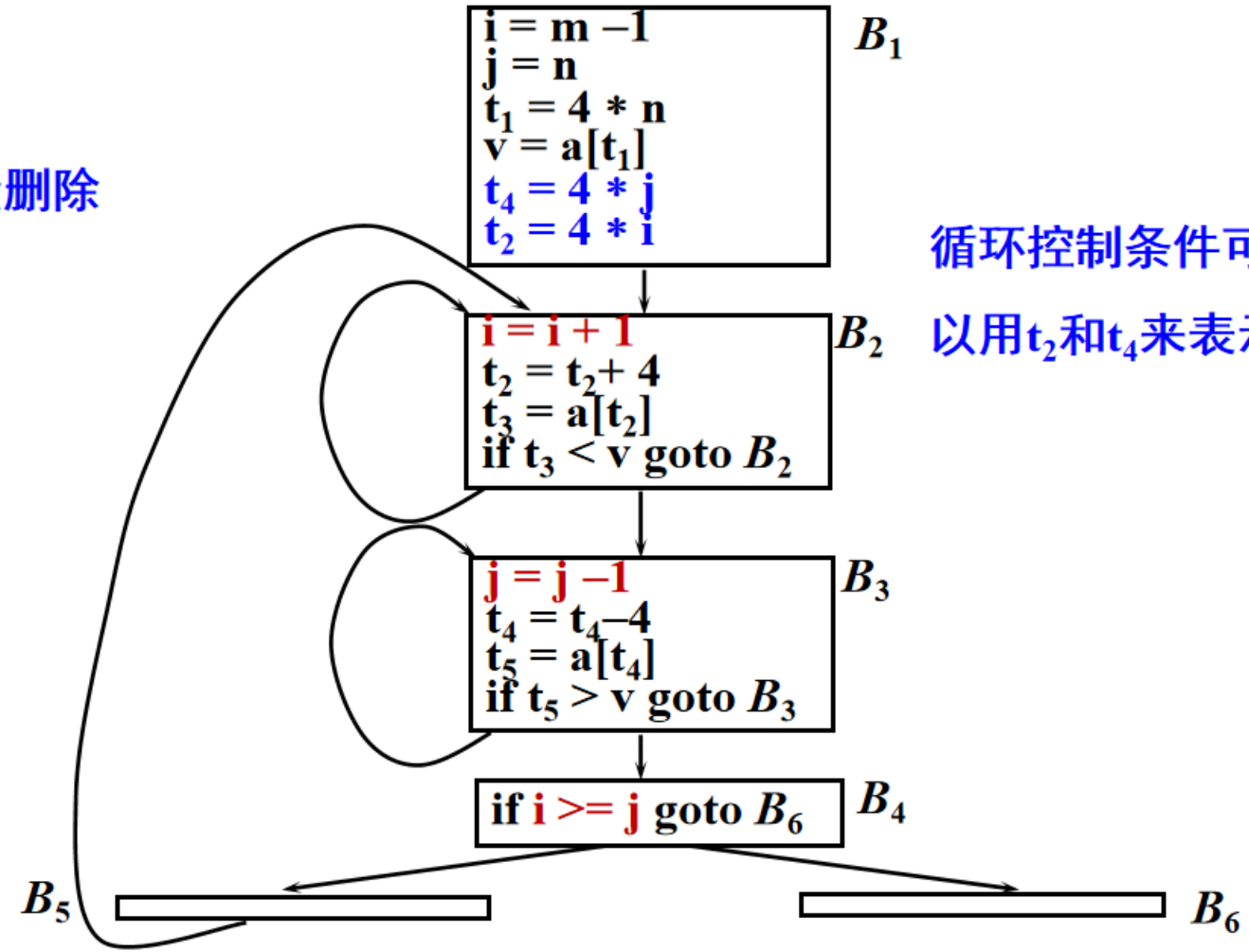
7.5 代码优化

强度削弱和归纳变量删除



7.5 代码优化

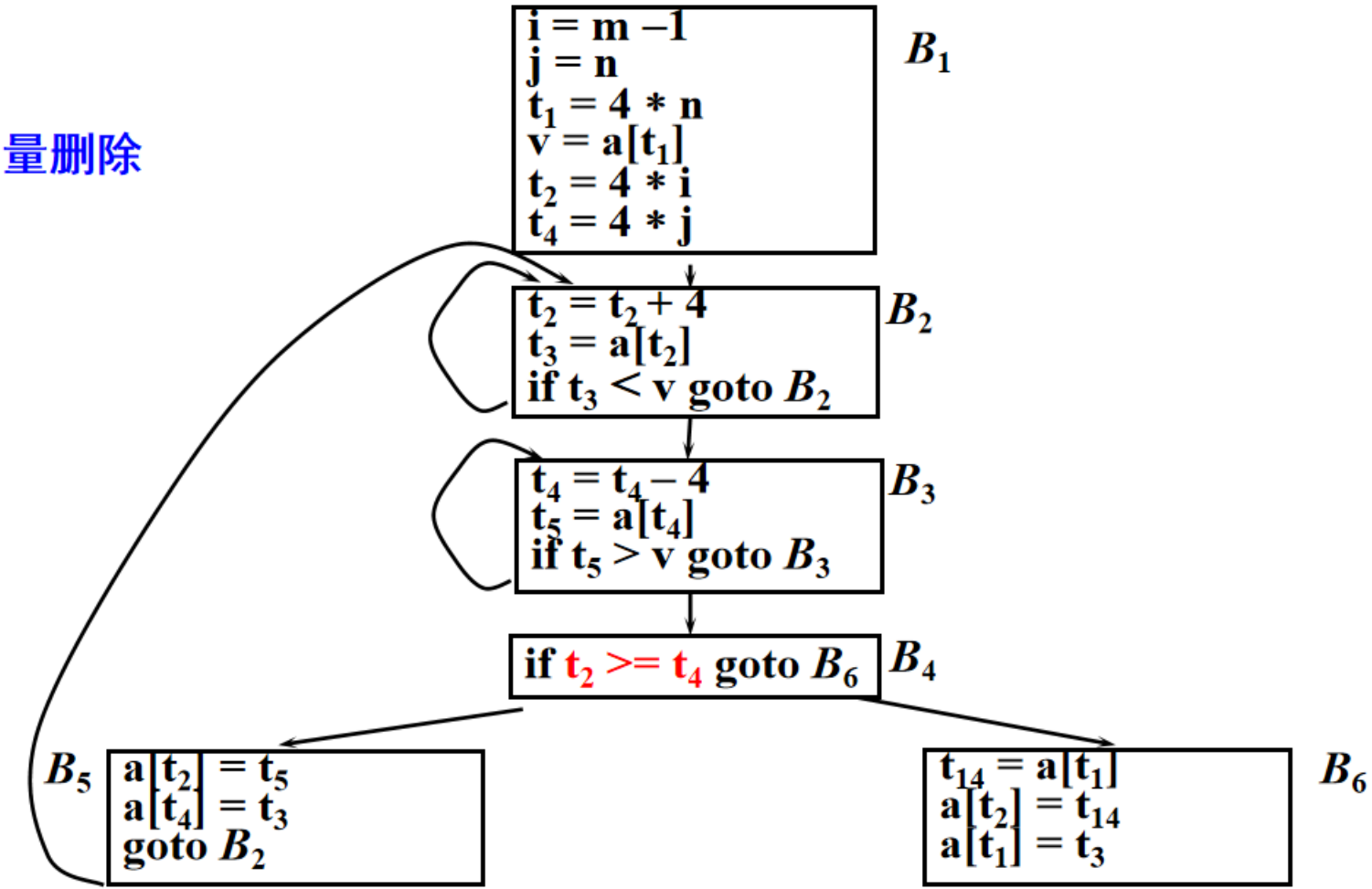
强度削弱和归纳变量删除



循环控制条件可以用t_2和t_4来表示

7.5 代码优化

强度削弱和归纳变量删除



7.6 目标代码生成

目标代码生成是编译器的最后一个阶段，其主要任务是将中间表示（Intermediate Representation, IR）转换为目标机器代码或汇编代码。这个过程涉及将高级语言的结构和语义映射到特定硬件架构的指令集上。目标代码生成的质量直接影响程序的执行效率和性能。

本课程用微型机的汇编指令来表示目标代码。

目标代码生成的基本概念

目标代码生成的目标是将中间表示转换为目标机器代码，通常包括以下几个方面：

① **指令选择**：选择适当的机器指令来实现中间表示中的操作。

② **寄存器分配**：将变量和临时值分配到可用的寄存器中。

③ **指令调度**：优化指令的执行顺序，以提高执行效率。

④ **生成目标代码**：将选择的指令和分配的寄存器组合成最终的目标代码。

目标代码生成的基本步骤

① 指令选择：根据中间表示中的操作，选择适当的机器指令。例如，如果中间表示中有一个加法操作，编译器需要选择相应的加法指令。

② 寄存器分配：在目标代码生成过程中，编译器需要将变量和临时值分配到可用的寄存器中。可以使用寄存器分配算法（如图着色算法）来优化寄存器的使用。

③ 指令调度：指令调度是指优化指令的执行顺序，以减少执行时间。编译器可以通过重新排列指令来避免数据依赖和提高并行性。

④ 生成目标代码：将选择的指令和分配的寄存器组合成最终的目标代码，通常以汇编语言或机器语言的形式输出。

7.6.1 指令选择

- 所谓指令选择是指寻找合适的目标机器指令序列来实现给定的中间代码。
- 目标机器指令系统的性质决定了指令选择的难易程度

  ① 指令系统的一致性和完备性是两个重要的因素

  ② 特殊机器指令的使用和指令速度是另一些重要的因素

假设有以下机器指令集，来讲解目标代码生成。

某机器的指令集

① 数据传输指令：这些指令用于在寄存器、内存和I/O端口之间传输数据。

MOV：将数据从一个位置移动到另一个位置。

PUSH：将数据压入栈。

POP：从栈中弹出数据。

IN：从I/O端口读取数据。

OUT：向I/O端口写入数据。

② 算术指令：这些指令用于执行算术运算。

ADD：加法。

SUB：减法。

MUL：无符号乘法。

IMUL：有符号乘法。

DIV：无符号除法。

IDIV：有符号除法。

某型机器的指令集

③ 逻辑指令：指令用于执行逻辑运算。

AND：按位与。

OR：按位或。

XOR：按位异或。

NOT：按位取反。

SHL：逻辑左移。

SHR：逻辑右移。

④ 控制指令：指令用于控制程序的执行流程。

JMP：无条件跳转。

CALL：调用子程序。

RET：从子程序返回。

INT：中断。

某型机器的指令集

⑤ 位操作指令：用于对位进行操作。

BT：测试位。

BTS：设置位。

BTR：清除位。

BTC：取反位。

⑥ 字符串操作指令：用于处理字符串。

MOVS：移动字符串。

CMPS：比较字符串。

SCAS：扫描字符串。

LODS：加载字符串。

STOS：存储字符串。

中间代码翻译成汇编程序代码的示例

- 中间代码翻译成汇编程序代码的示例

  t1 = a + b

  t2 = t1 * c

  d = t2 - 5

将这个中间代码翻译成8086汇编程序代码。

(1) 定义变量

- 首先，需要定义变量 a、b、c 和 d。
- 在8086汇编中，通常使用 .DATA 段来定义数据。

```
.DATA
a DB 10        ; 定义变量 a，初始值为 10
b DB 20        ; 定义变量 b，初始值为 20
c DB  5        ; 定义变量 c，初始值为 5
d DB  ?        ; 定义变量 d，初始值未定
```

(2) 汇编代码

- 将中间代码逐行翻译成如下汇编代码。

```
.CODE
MAIN PROC          ; 初始化数据段
 MOV AX, a         ;将变量 a 的值加载到寄存器 AX。
 ADD AX, b         ;将变量 b 的值加到 AX 中，计算 t1 = a + b。
 MOV BX, c         ;将变量 c 的值加载到寄存器 BX。
 MUL AX, BX        ;将 t1（在 AX 中）与 c（在 BX 中）相乘，结果存储在 AX 中（即 t2）。
 SUB AX, 5         ;从 t2 中减去 5，结果仍然存储在 AX 中。
 MOV d, AX         ;将最终结果存储到变量 d 的地址中。
RET                ; 结束程序
MAIN ENDP
END MAIN
```

寄存器分配

- 在编译技术中，寄存器分配是一个关键的优化步骤，旨在将程序中的变量和临时值有效地映射到有限数量的寄存器中。合理的寄存器分配可以显著提高程序的执行效率，减少对内存的访问。
- 寄存器分配的目标是将活跃变量（即在某个时刻被使用的变量）分配到可用的寄存器中。由于寄存器数量有限，编译器需要使用一些策略和算法来优化寄存器的使用。

寄存器分配方法

① 线性扫描分配算法

② 图着色分配算法

活跃变量分析

- 在程序的某个点上，如果一个变量在该点之后的某个地方被使用（即读取），则称该变量为“活跃变量”。相反，如果一个变量在某个点之后不再被使用，则称该变量为“非活跃变量”。

活跃变量分析的目的

① 消除冗余赋值：如果一个变量在赋值后不再被使用，则可以消除该赋值。

② 优化寄存器分配：通过识别活跃变量，编译器可以更有效地使用寄存器，减少内存访问。

③ 代码优化：通过识别活跃变量，编译器可以进行其他优化，如常量传播和死代码消除。

活跃变量分析的基本步骤

① 控制流图（CFG）的构建

首先，编译器需要构建程序的控制流图（Control Flow Graph, CFG）。

CFG是一个有向图，其中节点表示基本块，边表示控制流的转移。

② 定义变量的使用和定义

在CFG中，定义每个基本块的活跃变量集合。活跃变量集合是一个变量的集合，表示在该基本块的结束处仍然活跃的变量。

③ 迭代计算：通过迭代的方式计算每个基本块的活跃变量集合，直到集合不再变化为止。通常使用以下公式：

输入集合：**LiveIn[B]** 表示基本块 B 开始时的活跃变量集合。

输出集合：**LiveOut[B]** 表示基本块 B 结束时的活跃变量集合。

公式如下：

$$\mathbf{LiveIn(B)= USE(B) \cup (LiveOut(B)- DEF(B))}$$

$$\mathbf{LiveOut(B)} = \cup_{s \in successors(B)} \mathrm{LiveIn}(s)$$

其中：

USE[B] 是在基本块 B 中使用的变量集合；

DEF[B] 是基本块 (B) 中所有赋值的变量集合；

successors(B) 是基本块 (B) 的所有后继块。

计算中间代码中的活跃变量的示例

- 将通过一个简单的中间代码片段，构建控制流图（CFG），并逐步计算每个基本块的活跃变量。

① t1 = a + b

② t2 = t1 * c

③ d = t2 - 5

④ return d

（1）构建控制流图（CFG）

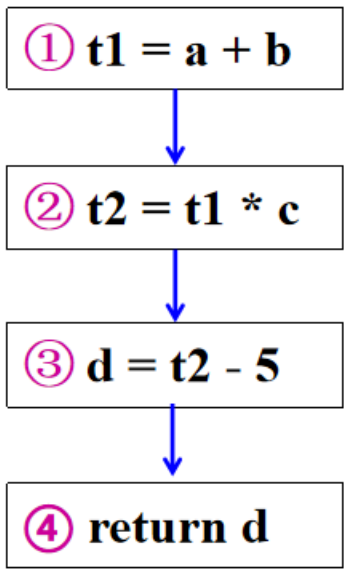
- 在这个例子中，整个代码可以视为一个基本块，因为它没有分支或跳转。可以将其表示为一个简单的控制流图：

① t1 = a + b

② t2 = t1 * c

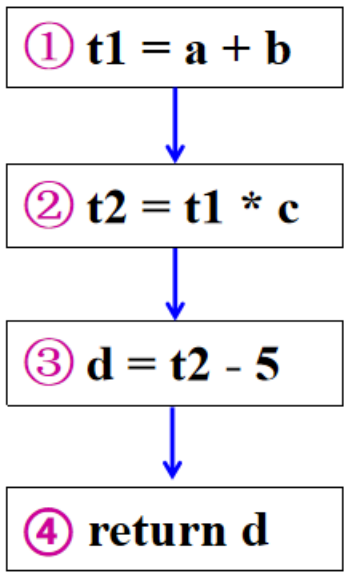
③ d = t2 - 5

④ return d



（2）定义变量的使用和定义

- 在每个基本块中，可以定义变量的使用（USE）和定义（DEF）集合：



•USE集合：

USE[1] = {a, b}（使用了变量 a 和 b）

USE[2] = {t1, c}（使用了变量 t1 和 c）

USE[3] = {t2}（使用了变量 t2）

USE[4] = {d}

•DEF集合：

DEF[1] = {t1}（定义了变量 t1）

DEF[2] = {t2}（定义了变量 t2）

DEF[3] = {d}（定义了变量 d）

DEF[4] = { }

计算活跃变量的算法

(1) 初始化活跃变量集合：对于每个基本块，初始化活跃变量集合为一个空集合。

(2) 反向遍历控制流图：从程序的出口基本块开始，反向遍历控制流图，直到入口基本块。对于每个基本块，更新活跃变量集合。

(3) 对于每个基本块 B，计算活跃变量集合 LiveIn[B],LiveOut[B]：

(4) 使用变量：将基本块中所有使用的变量USE(B)加入 LiveIn[B]。

(5) 赋值变量：从 LiveIn[B] 中删除基本块中所有赋值的变量。

(6) 合并后继块的活跃变量：将所有后继块的活跃变量集合合并到 LiveOut[B] 中。

(7) 重复步骤3 和 6，直到活跃变量集合不再变化（即收敛）。

初始化活跃变量集合

- 将使用以下集合来表示每个基本块的活跃变量：

  LiveIn[B]： 表示在基本块 B 开始时的活跃变量集合。

  LiveOut[B]： 表示在基本块 B 结束时的活跃变量集合。

初始状态为：LiveIn[1] = {}（没有活跃变量）

LiveIn[2] = {}（没有活跃变量）

LiveIn[3] = {}（没有活跃变量）

LiveIn[4] = {}（没有活跃变量）

LiveOut[1] = {}（没有活跃变量）

LiveOut[2] = {}（没有活跃变量）

LiveOut[3] = {}（没有活跃变量）

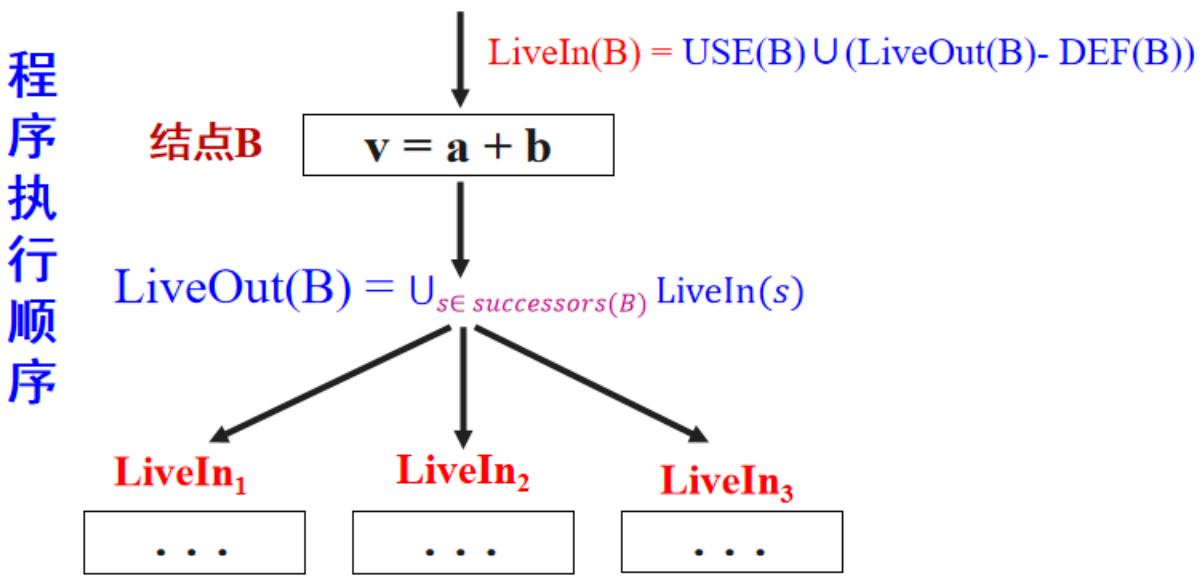
LiveOut[4] = {d}（假设d变量需要返回给调用程序）

迭代计算活跃变量集合

- 将根据公式进行迭代计算：

$$\mathbf{LiveOut(B) = \bigcup_{s\in\, successors(B)} LiveIn(s)}$$

$$\mathbf{LiveIn(B) = USE(B) \cup (LiveOut(B) - DEF(B))}$$



第1次迭代计算活跃变量

- 将根据公式进行迭代计算：

$$LiveIn(B) = USE(B) \cup (LiveOut(B) - DEF(B))$$

$$LiveOut(B) = \cup_{s \in successors(B)} LiveIn(s)$$

① 计算：$LiveOut_1[4] = \{d\}$

$LiveIn_1[4] = \{d\} \cup \{d\} - \{\} = \{d\}$

② 计算：$LiveOut_1[3] = LiveIn_1[4] = \{d\}$

$LiveIn_1[3] = \{t2\} \cup \{d\} - \{d\} = \{t2\}$

③ 计算：$LiveOut_1[2] = LiveIn_1[3] = \{t2\}$

$LiveIn_1[2] = \{t1,c\} \cup \{t2\} - \{t2\} = \{t1,c\}$

④ 计算：$LiveOut_1[1] = LiveIn_1[2] = \{t1,c\}$

$LiveIn_1[1] = \{a,b\} \cup \{t1,c\} - \{t1\} = \{a,b,c\}$

•USE集合：

USE[1] = {a, b}（使用了变量 a 和 b）

USE[2] = {t1, c}（使用了变量 t1 和 c）

USE[3] = {t2}（使用了变量 t2）

USE[4] = {d}

•DEF集合：

DEF[1] = {t1}（定义了变量 t1）

DEF[2] = {t2}（定义了变量 t2）

DEF[3] = {d}（定义了变量 d）

DEF[4] = { }

第2次迭代计算活跃变量

- 将根据公式进行迭代计算：

$$LiveIn(B) = USE(B) \cup (LiveOut(B) - DEF(B))$$

$$LiveOut(B) = \cup_{s \in successors(B)} LiveIn(s)$$

① 计算：$LiveOut_2[4] = \{d\}$

$LiveIn_2[4] = \{d\} \cup \{d\} - \{\} = \{d\}$

② 计算：$LiveOut_2[3] = LiveIn_2[4] = \{d\}$

$LiveIn_2[3] = \{t2\} \cup \{d\} - \{d\} = \{t2\}$

③ 计算：$LiveOut_2[2] = LiveIn_2[3] = \{t2\}$

$LiveIn_2[2] = \{t1,c\} \cup \{t2\} - \{t2\} = \{t1,c\}$

④ 计算：$LiveOut_2[1] = LiveIn_2[2] = \{t1,c\}$

$LiveIn_2[1] = \{a,b\} \cup \{t1,c\} - \{t1\} = \{a,b,c\}$

•USE集合：

USE[1] = {a, b}（使用了变量 a 和 b）

USE[2] = {t1, c}（使用了变量 t1 和 c）

USE[3] = {t2}（使用了变量 t2）

USE[4] = {d}

•DEF集合：

DEF[1] = {t1}（定义了变量 t1）

DEF[2] = {t2}（定义了变量 t2）

DEF[3] = {d}（定义了变量 d）

DEF[4] = { }

活动变量

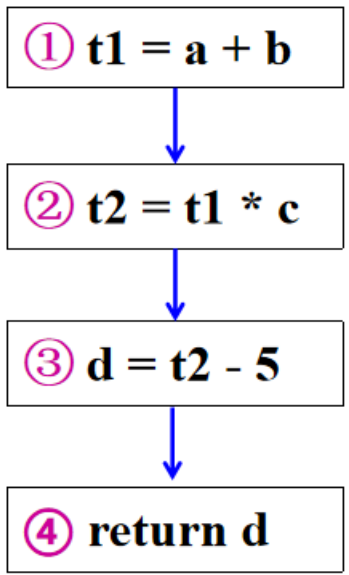
- 经过迭代，最终得到每个基本块的活跃变量集合：

① LiveIn[1] = {a,b,c}， LiveOut[1] = {t1,c}

② LiveIn[2] = {t1,c}， LiveOut[2] = {t2}

③ LiveIn[3] = {t2}， LiveOut[3] = {d}

④ LiveIn[4] = LiveOut[4] = {d}



构建活跃变量的相干图

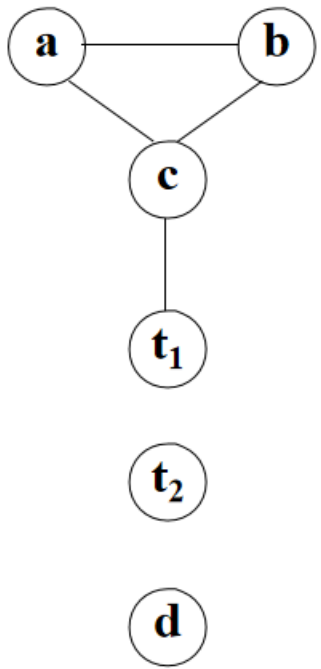
① LiveIn[1] = {a,b,c}，LiveOut[1] = {t1,c}

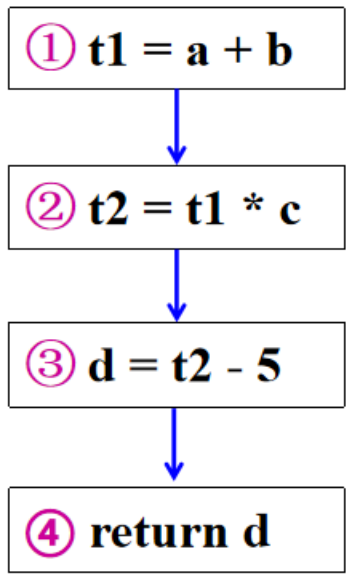
② LiveIn[2] = {t1,c}，LiveOut[2] = {t2}

③ LiveIn[3] = {t2}，LiveOut[3] = {d}

④ LiveIn[4] = LiveOut[4] = {d}

- 以活跃变量作为图的顶点
- 在同一活跃变量集的变量不能放在相同寄存器，因此，在相干图中连接一条边。





图的k着色问题

图的 k 着色是图论中的重要问题，旨在为图的每个节点分配颜色，使得相邻的节点具有不同的颜色，并且使用的颜色数量不超过 k。这个问题在许多应用中都有重要意义，例如在寄存器分配、任务调度和地图着色等领域。

图的k着色算法

① 如果图中存在度数小于k-1的结点，输出该结点到列表list中；否则，转向③；

② 删除与该结点相连的边后的子图G；重复步骤①

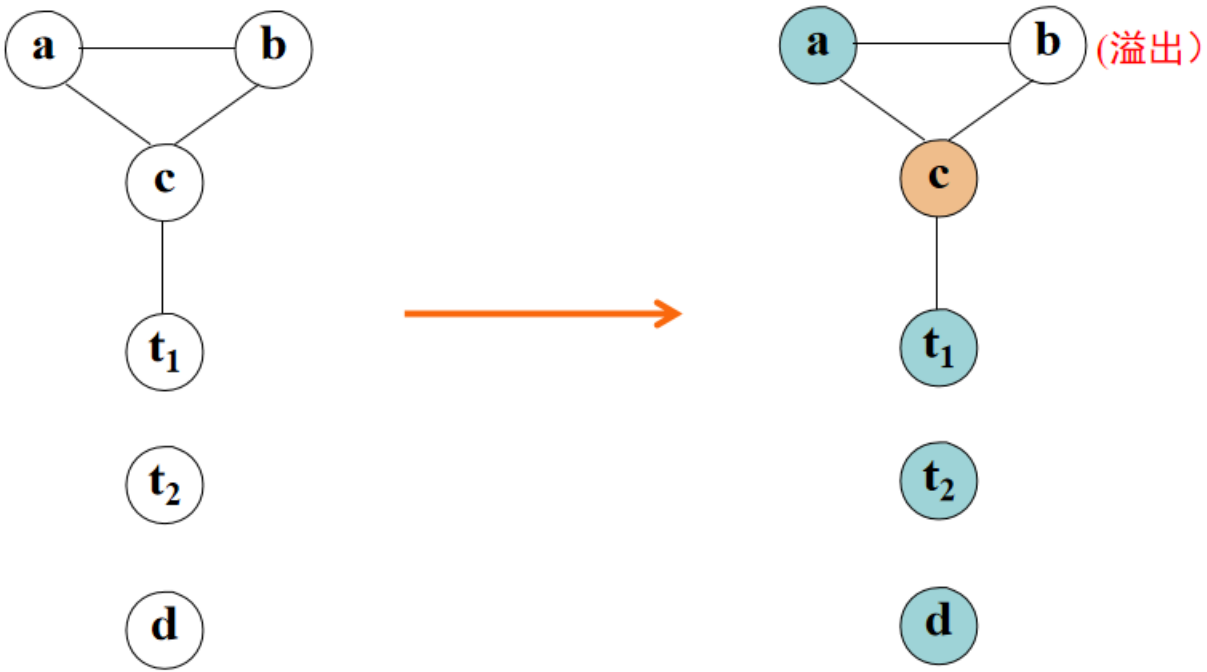
③ 去掉子图G中不重要的结点及相连的边，直到出现度数小于k-1的结点，转向①。

④ 在列表list中的结点，按照逆序为当前节点分配一种未被邻居结点使用的颜色。

⑤ 重复④：继续处理下一个节点，直到list中所有结点都被着色。

例如: k=2的图着色

① list列表： d t_2 t_1 a c



① t1 = a + b

② t2 = t1 * c

③ d = t2 - 5

④ return d

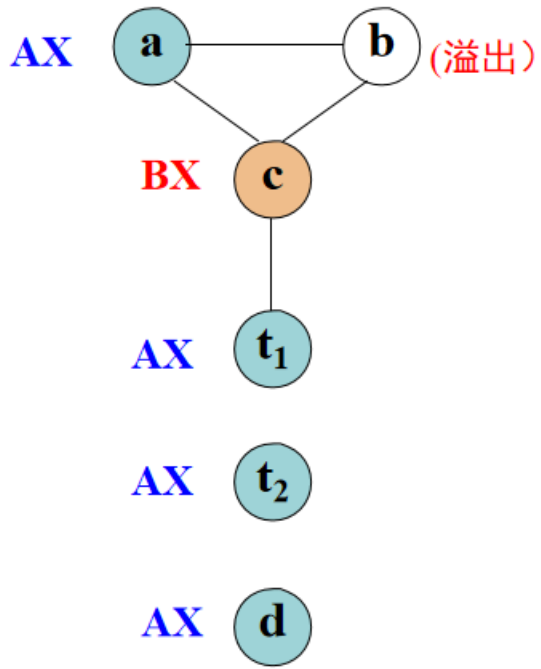
k个寄存器的分配算法

① 采用图k着色算法给图中的结点被着色；

② 将k个寄存器分别表示k种不同的颜色；

③ 根据图中着色结点对应的变量分配给颜色对应的寄存器；

④ 没有被着色的结点对应的变量转存（溢出）到内存中。



根据寄存器分配，可以生成目标代码。

```
; 假设 a, b, c, d 存储在内存中;
MOV AX, a          ;将变量 a 的值加载到寄存器 AX。
ADD AX, b          ;将变量 b 的值加到 AX 中，计算 t1 = a + b。
MOV BX, c          ;将变量 c 的值加载到寄存器 BX。
MUL AX, BX         ;将 t1（在 AX 中）与 c（在 BX 中）相乘，结果存储在 AX 中（即 t2）。
SUB AX, 5          ;从 t2 中减去 5，结果仍然存储在 AX 中。
MOV d, AX          ;将最终结果存储到变量 d 的地址中。
RET                ; 结束程序
```

线性扫描寄存器分配算法

第一步：前置分析

1. 活跃性分析：解活跃变量的数据流方程，得到每个活跃变量v的def 集合与use集合。

2. 构造活跃区间：对每个活跃变量 v，计算

① **start(v) = v 第一次 def 的指令序号；**

② **end(v) = v 最后一次 use 的指令序号。**

3. 排序：把所有活跃区间按 start(v) 升序排列，得到序列 $I = [I_1, I_2, \ldots, I_n]$。所有区间按起点排队；k 个物理寄存器全部空闲；"正在占用寄存器" 的列表list为空。

构造活跃区间的示例

1. 原始 IR（三地址码，k = 2，物理寄存器 r0、r1）

#	指令	def	use
1	a = 0	a	—
2	b = 1	b	—
3	c = a + b	c	a, b
4	d = c * 2	d	c
5	e = d + b	e	d, b
6	f = e + a	f	e, a
7	print f	—	f
8	print e	—	e



2. 活跃性分析结果（从后往前推）

变量	start (首次 def)	end (最后 use)	变量的活跃区间
a	1	6	a [1,6]
b	2	5	b [2,5]
c	3	4	c [3,4]
d	4	5	d [4,5]
e	5	8	e [5,8]
f	6	7	f [6,7]

线性扫描寄存器分配算法

第二步：分配寄存器

① 回收在当前区间开始前已经结束的旧区间所占的物理寄存器。

② 寄存器分配：

a) 在当前指令的执行时，查看活跃变量列表，检查当前寄存器是否足够用。

b) 如果寄存器足够，分配寄存器给新变量。

c) 如果寄存器不足，比较使用寄存器的变量列表里活得最久的变量和当前新来变量的谁活得更长，把活得久的挤到栈上、空出寄存器给对方（如果新来的活得更长，就挤新来的自己）。

③ 更新状态：更新活跃变量列表，被挤到栈上的变量，在它每次赋值之后存一次栈、每次使用之前取一次栈。

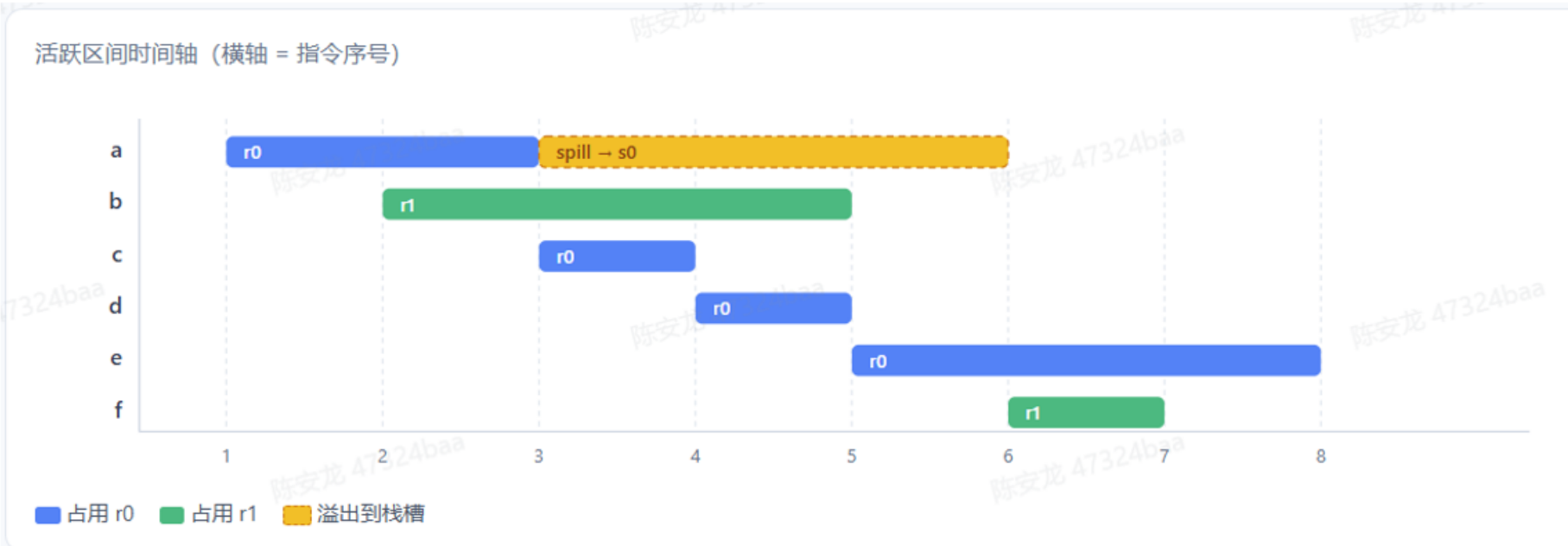
④ 整趟扫描只走一遍，不回头重做活跃性分析。

线性扫描寄存器分配算法示例

k = 2（物理寄存器 r0、r1），所有变量按 start 排序后线性扫描；

步骤	处理区间	expire 回收	active 满?	决策	分配结果
1	a [1,6]	—	0/2 否	分配寄存器	a → r0
2	b [2,5]	—	1/2 否	分配寄存器	b → r1
3	c [3,4]	—	2/2 是	a.end=6 > c.end=4，溢出 a，c 接管 r0	a → s0；c → r0
4	d [4,5]	c.end=4， 释放 r0	1/2 否	分配寄存器	d → r0
5	e [5,8]	b.end=5、d.end=5，释放 r1、r0	0/2 否	分配寄存器	e → r0
6	f [6,7]	—	1/2 否	分配寄存器	f → r1

线性扫描寄存器分配算法图示



生成的目标代码

① a = 0

② b = 1

③ c = a + b

④ d = c * 2

⑤ e = d + b

⑥ f = e + a

⑦ print f

⑧ print e

根据寄存器分配得出的结果，生成目标代码的指令如下：

```
movi   r0, #0       ; 1:  a = 0        (a → r0)
movi   r1, #1       ; 2:  b = 1        (b → r1)

str    r0, s0       ; 3.1: 溢出a       r0 即将被 c 覆盖
add    r0, r0, r1   ; 3.2: c = a + b   (c → r0)

movi   r2, #2       ; 4.1: 立即数 2
mul    r0, r0, r2   ; 4.2: d = c * 2    (d → r0)

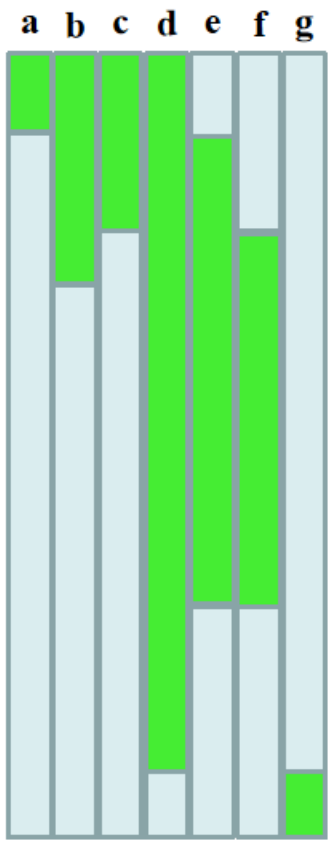
add    r0, r0, r1   ; 5:   e = d + b    (e → r0；b 死，r1 回收)

ldr    r2, s0       ; 6.1: 装入 a      a 最后一次被 use
add    r1, r0, r2   ; 6.2: f = e + a    (f → r1)

print  r1           ; 7:   print f     (f 在 r1)
print  r0           ; 8:   print e     (e 在 r0)
```

寄存器的图着色分配示例2

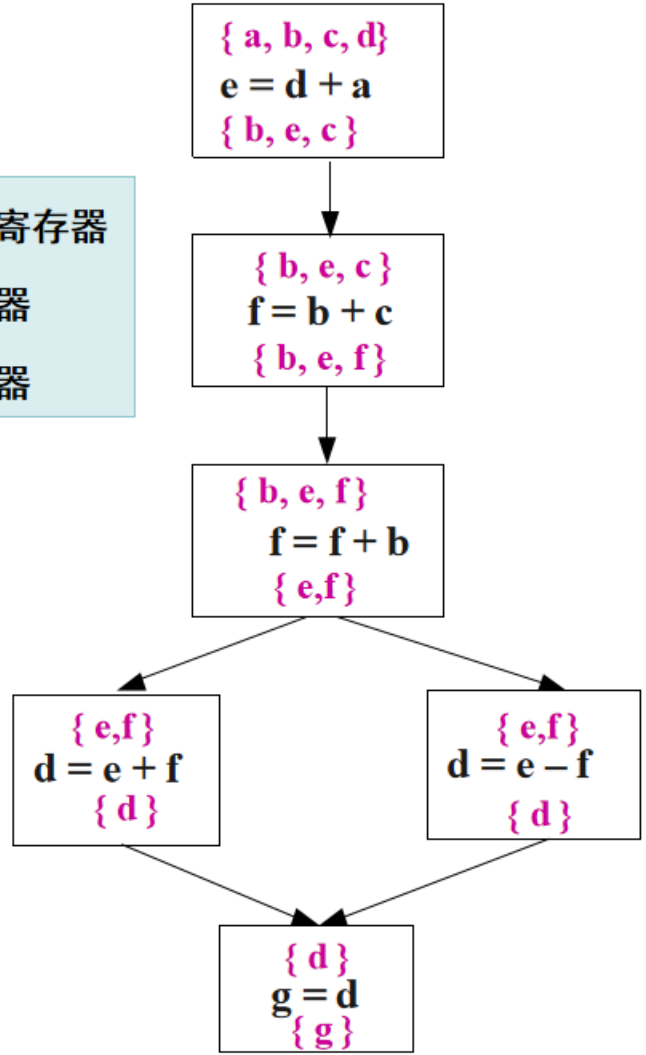
e = d + a
f = b + c
f = f + b
IfZ e Goto _L0
d = e + f
Goto _L1;
_L0:
d = e - f
_L1:
g = d



根据变量的活跃区间分配寄存器

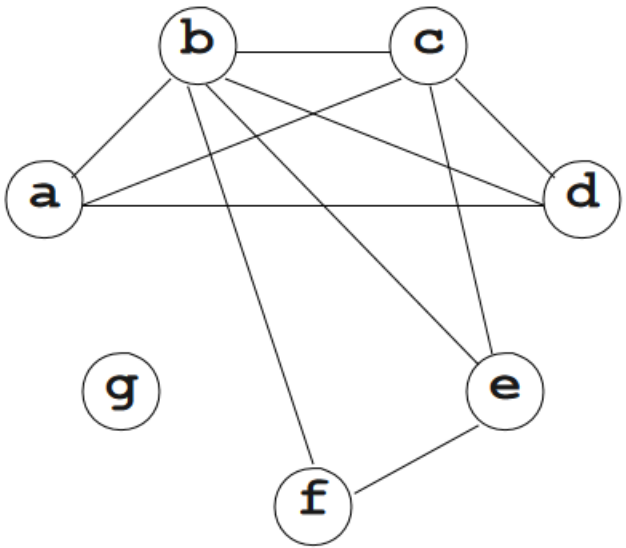
活跃区间开始：分配寄存器

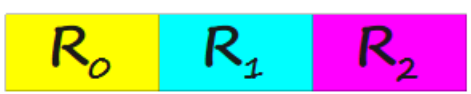
活跃区间结束：回收寄存器

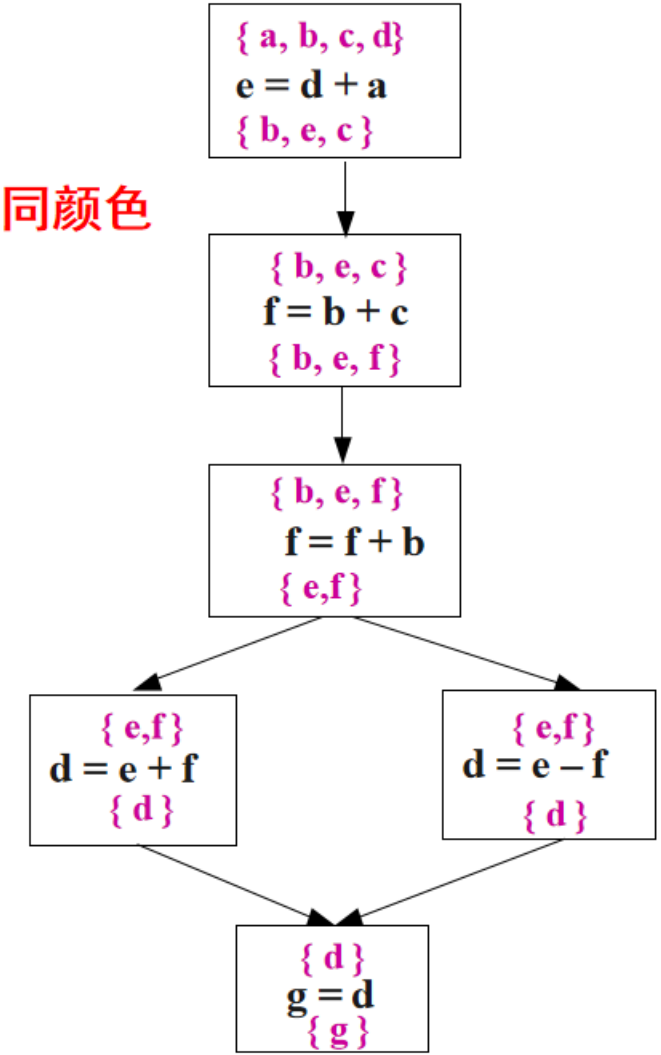


寄存器的图着色分配示例2

- 构建活跃变量相干图，变量将被分配与寄存器相同颜色

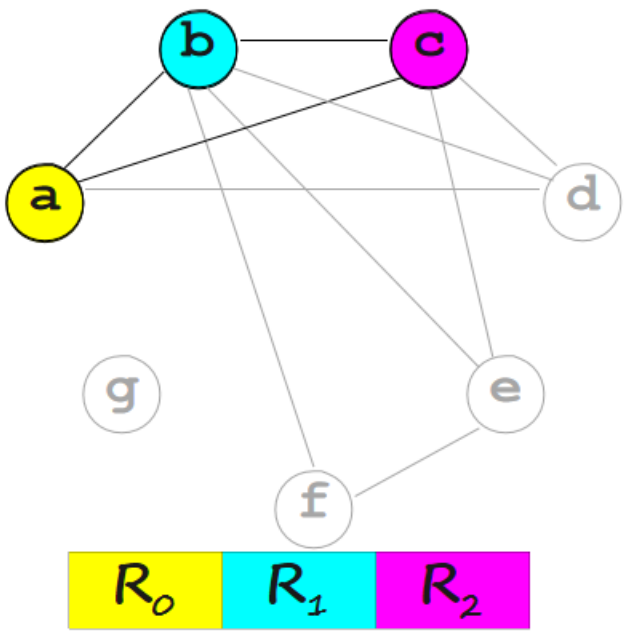


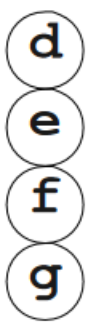


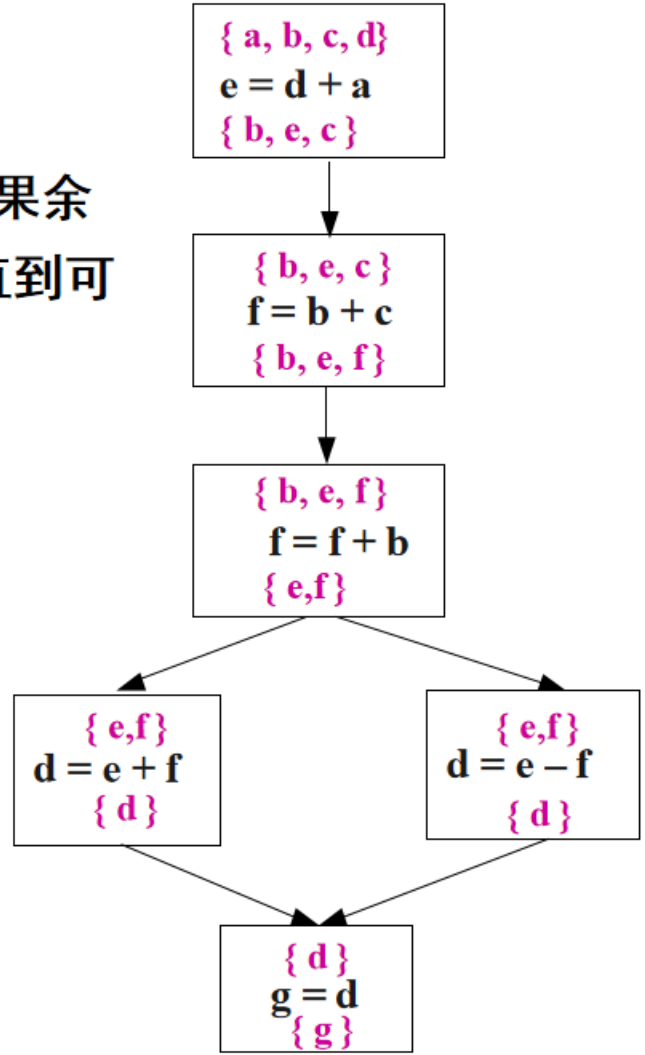


寄存器的图着色分配示例2

- 运用图k着色算法，删除度数小于k结点及边，如果余下子图结点度数均大于k，去掉不重要的结点，直到可以用k着色为止。

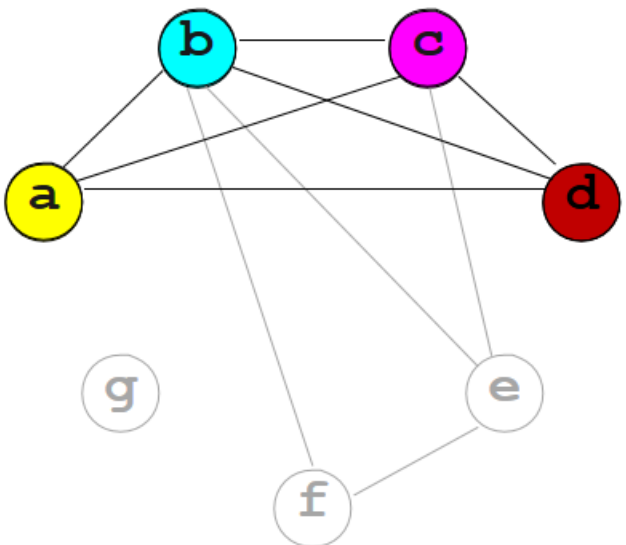




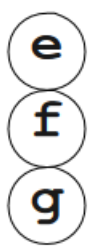


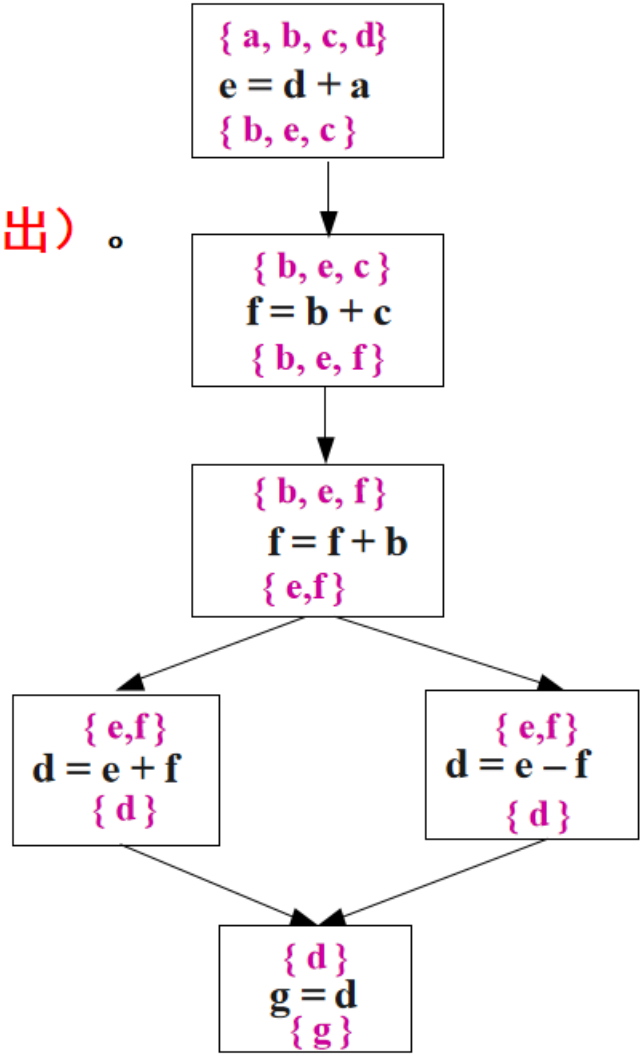
寄存器的图着色分配示例2

- 子图中加回度数大于k的结点，并转存到内存（溢出）。



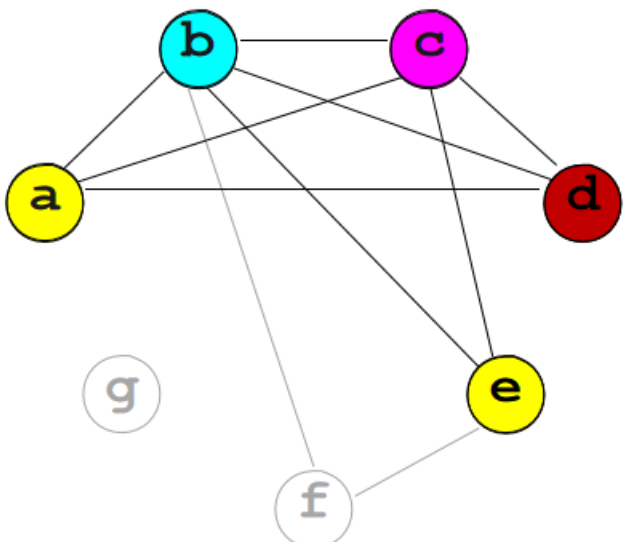
(溢出)





寄存器的图着色分配示例2

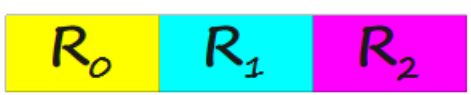
- 加回子图中度数小于k的结点，并用不同于相邻顶点的颜色着色。

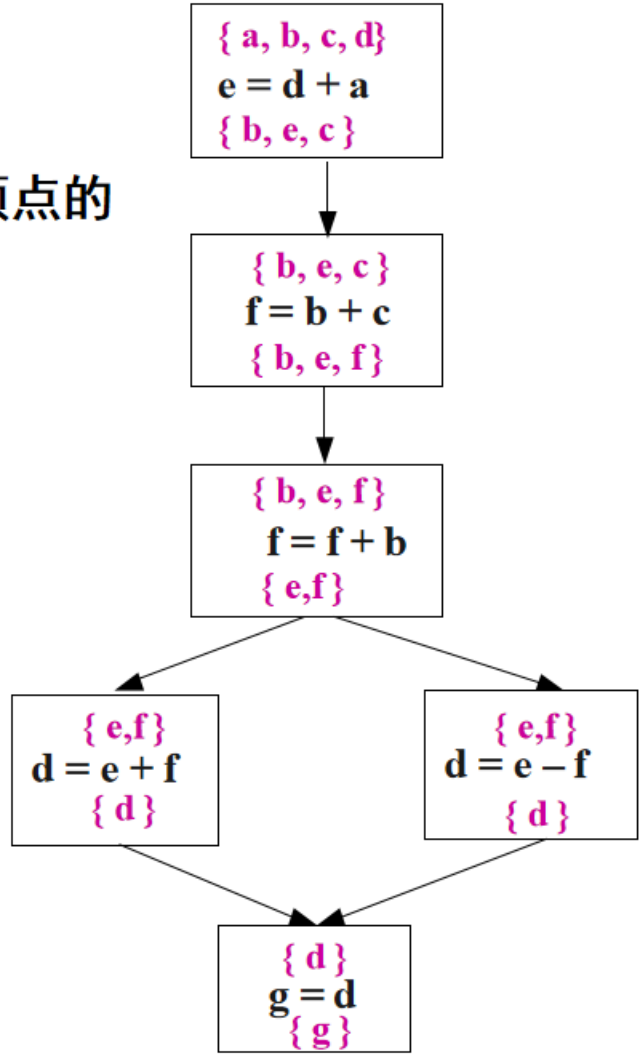


(溢出)

f

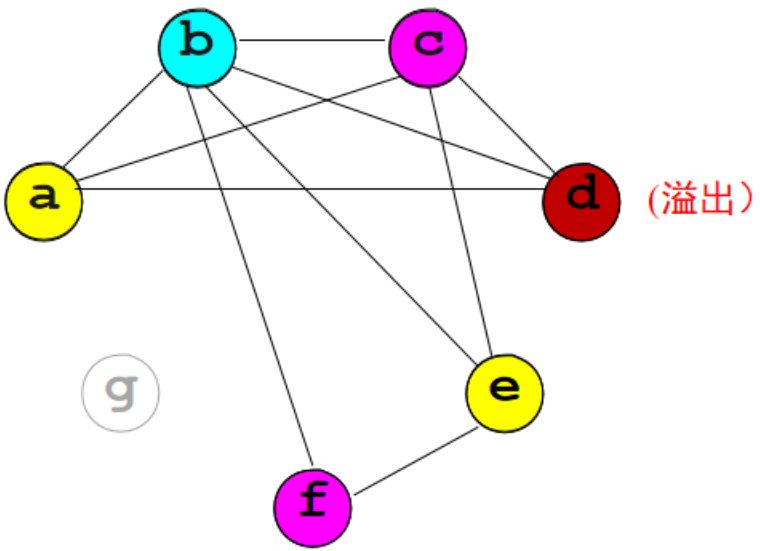
g

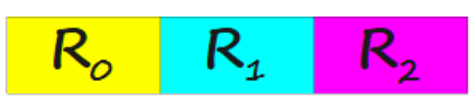




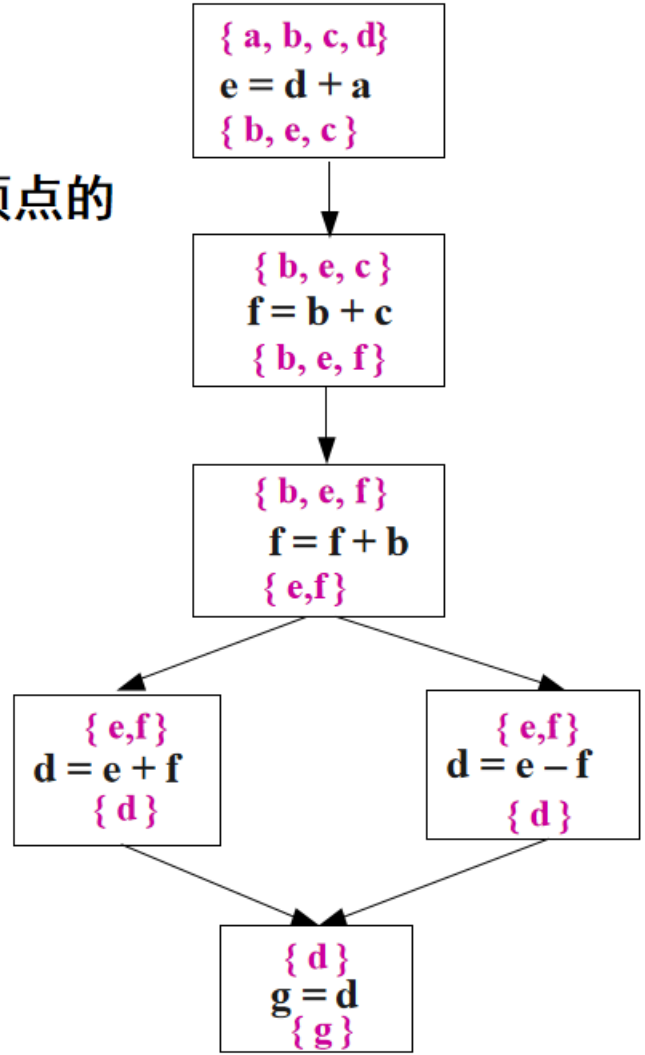
寄存器的图着色分配示例2

- 加回子图中度数小于k的结点，并用不同于相邻顶点的颜色着色。



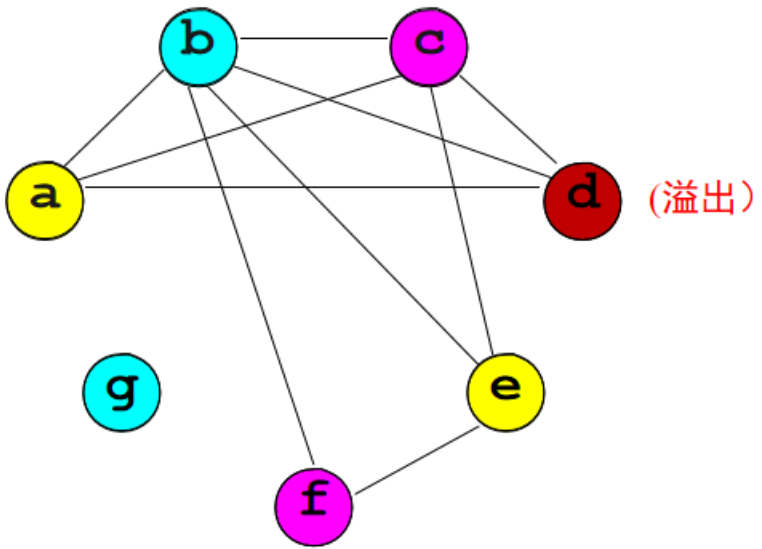


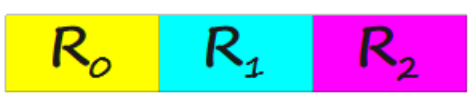
g

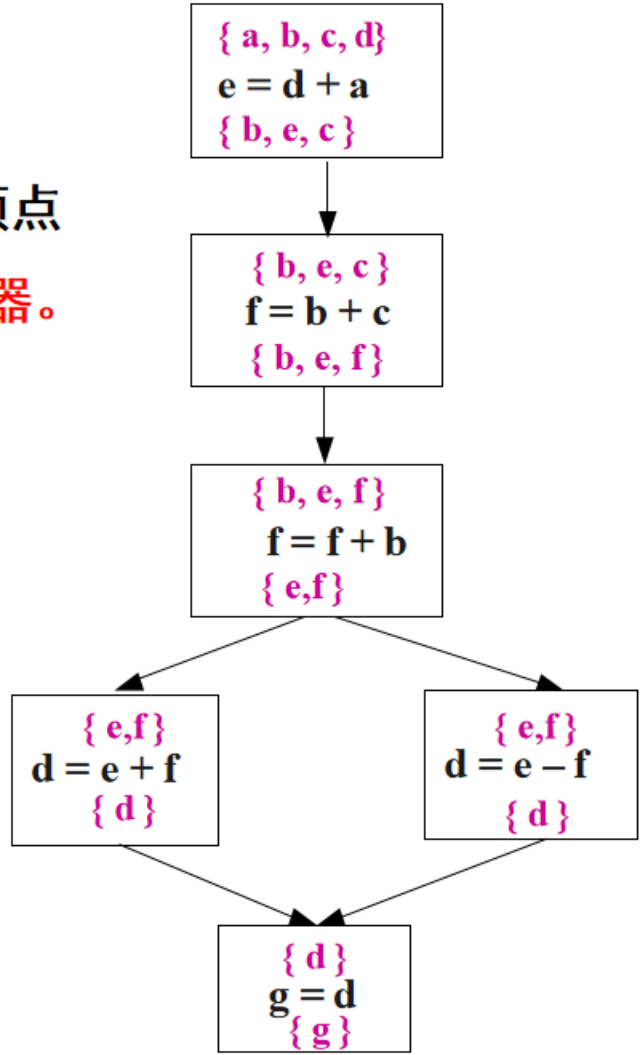


寄存器的图着色分配示例2

- 加回子图中度数小于k的结点，并用不同于相邻顶点的颜色着色，下图变量被分配到相同颜色的寄存器。







谢谢大家！